

ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO TIỂU LUẬN
HỌC PHẦN: CƠ SỞ DỮ LIỆU NÂNG CAO
ĐỀ TÀI: ỨNG DỤNG CƠ SỞ DỮ LIỆU MONGODB VÀO BÀI TOÁN
QUẢN LÝ CHI TIÊU THÔNG MINH

Giảng viên hướng dẫn :

TS. Phạm Văn Hà

Học viên thực hiện

Nhóm 03

Lê Hoàn - 2025700249

Nguyễn Nhật Minh - 2025700346

Doãn Mạnh Hiếu - 2025700305

Khoá

CH HTTT K15.3

Mã lớp

20252IT7305001

Mã học phần

IT7305

Hà Nội, tháng 8 năm 2026

LỜI NÓI ĐẦU

Với sự trân trọng và lòng biết ơn vô cùng, chúng tôi xin gửi lời cảm ơn chân thành đến quý thầy/cô Trường Công nghệ thông tin và Truyền thông – Đại học Công Nghiệp Hà Nội đã trao cho chúng tôi những kiến thức nền tảng để chúng tôi có thể hoàn thiện bài tiểu luận này. Và trên hết, chúng tôi xin gửi lời cảm ơn sâu sắc đến TS. Phạm Văn Hà đã trực tiếp nhiệt tình và tận tâm hướng dẫn chúng tôi thực hiện đề tài.

Mặc dù đã đầu tư nhiều thời gian để tìm hiểu tuy nhiên bài viết có thể sẽ còn nhiều thiếu sót. Chúng tôi mong rằng sẽ nhận được những lời góp ý của quý thầy cô để bài viết hoàn thiện hơn.

Hà Nội, ngày 30 tháng 8 năm 2026

Đại diện nhóm

(Ký ghi và rõ họ tên)

DANH MỤC CÁC THUẬT NGỮ, KÝ HIỆU VÀ CÁC CHỮ VIẾT TẮT

Các chữ viết tắt được sắp xếp theo thứ tự ABC, mỗi chữ viết tắt được giải nghĩa đầy đủ tại lần xuất hiện đầu tiên trong báo cáo.

Chữ viết tắt	Giải nghĩa
ACID	Atomicity, Consistency, Isolation, Durability — các thuộc tính toàn vẹn giao dịch
API	Application Programming Interface — giao diện lập trình ứng dụng
BSON	Binary JSON — định dạng dữ liệu nhị phân của MongoDB
CAP	Consistency, Availability, Partition tolerance — định lý CAP trong hệ phân tán
EDU	Hũ Giáo dục & Nâng cao Kỹ năng (Education) — một trong 6 Hũ tài chính
ESR	Equality – Sort – Range — quy tắc thiết kế compound index của MongoDB
FFA	Hũ Tự do Tài chính (Financial Freedom Account) — một trong 6 Hũ tài chính
GIVE	Hũ Cho đi & Thiện nguyện (Give) — một trong 6 Hũ tài chính
JSON	JavaScript Object Notation — định dạng trao đổi dữ liệu dạng văn bản
JWT	JSON Web Token — chuẩn mã hóa token dùng cho xác thực phiên
LTSS	Hũ Tiết kiệm Dài hạn (Long-Term Savings) — một trong 6 Hũ tài chính
NEC	Hũ Nhu cầu Thiết yếu (Necessities) — một trong 6 Hũ tài chính
NLP	Natural Language Processing — xử lý ngôn ngữ tự nhiên
PLAY	Hũ Hưởng thụ & Giải trí (Play) — một trong 6 Hũ tài chính

RAM	Random Access Memory — bộ nhớ truy cập ngẫu nhiên
REST / RESTful	Representational State Transfer — phong cách kiến trúc API theo Fielding (2000)
SPA	Single Page Application — ứng dụng web một trang
TF-IDF	Term Frequency – Inverse Document Frequency — trọng số đặc trưng văn bản
TTL	Time To Live — thời gian sống của một bản ghi dữ liệu trước khi tự động bị xóa

DANH MỤC HÌNH ẢNH

Hình 2.1. Sơ đồ nguyên lý vận hành và phân bổ dòng tiền mô hình 6 Hồ tài chính	13
Hình 2.2. Sơ đồ mô hình quan hệ giữa 8 Collections trong CSDL MongoDB....	15
Hình 2.3. Sơ đồ luồng xử lý đa nhánh của MongoDB \$facet Aggregation Pipeline	20
Hình 3.1. Sơ đồ kiến trúc tổng thể 3-Tier của hệ thống Smart Expense.....	27
Hình 3.2. Quy trình xử lý NLP và suy luận học máy trong AI Microservice	28
Hình 3.3. Sơ đồ tuần tự quy trình tạo giao dịch thông minh kết hợp AI & Anomaly Detection	32
Hình 3.4. Sơ đồ tuần tự tự động hóa thực thi giao dịch định kỳ đến hạn	33
Hình 3.5. Giao diện biểu đồ trực quan hóa phân bổ 6 Hồ và biến động thu chi..	35

DANH MỤC BẢNG BIỂU

Bảng 1.1. So sánh ưu điểm và nhược điểm của MongoDB	8
Bảng 2.1. Bảng tổng hợp 8 Collections và quan hệ tham chiếu	16
Bảng 2.2. Bảng tổng hợp chiến lược Indexing tối ưu cho 8 Collections	19
Bảng 3.1. Tổng hợp stack công nghệ và môi trường triển khai hệ thống	26
Bảng 3.2. Bảng phân tích chi tiết hiệu năng phân loại đa chiều của mô hình NLP30	
Bảng 3.3. Bảng tổng hợp 38 API Endpoints theo 8 nhóm nghiệp vụ	33
Bảng 3.4. Bảng tổng hợp kết quả kiểm thử chức năng và tích hợp hệ thống	36
Bảng 3.5. Kết quả kiểm thử tải trọng và phân vị độ trễ hệ thống với k6	37
Bảng 3.6. Bảng dự toán chi phí hạ tầng và vận hành hàng tháng của hệ thống ..	38

MỤC LỤC

LỜI NÓI ĐẦU	i
DANH MỤC CÁC THUẬT NGỮ, KÝ HIỆU VÀ CÁC CHỮ VIẾT TẮT	ii
DANH MỤC HÌNH ẢNH	iv
DANH MỤC BẢNG BIỂU	v
MỤC LỤC.....	vi
MỞ ĐẦU.....	1
1. LÝ DO LỰA CHỌN ĐỀ TÀI	1
2. MỤC TIÊU NGHIÊN CỨU	1
3. PHẠM VI NGHIÊN CỨU	2
4. PHƯƠNG PHÁP NGHIÊN CỨU	3
5. CẤU TRÚC BÁO CÁO	3
CHƯƠNG 1. TỔNG QUAN VỀ NOSQL VÀ MONGODB	5
1.1. TỔNG QUAN VỀ CƠ SỞ DỮ LIỆU NOSQL	5
1.1.1. Khái niệm cơ bản về cơ sở dữ liệu NoSQL.....	5
1.1.2. Đặc điểm và phân loại cơ sở dữ liệu NoSQL	5
1.2. TỔNG QUAN VỀ MONGODB	7
1.2.1. Giới thiệu về MongoDB.....	7
1.2.2. Các thành phần cơ bản của MongoDB	7
1.2.3. So sánh ưu điểm và nhược điểm của MongoDB.....	8
1.3. TỔNG KẾT CHƯƠNG 1	9
CHƯƠNG 2. THIẾT KẾ CƠ SỞ DỮ LIỆU NÂNG CAO.....	10
2.1. Phân tích yêu cầu dữ liệu.....	10
2.1.1. Bối cảnh nghiên cứu và các tác nhân hệ thống	10
2.1.2. Phân tích cấu trúc dữ liệu 8 phân hệ nghiệp vụ cốt lõi.....	10
2.1.3. Mô hình 6 hồ chỉ tiêu và cơ chế quản trị dòng tiền thông minh.....	11
2.2. Định lý CAP và bản chất hệ phân tán của MongoDB.....	13
2.3. Mô hình hóa dữ liệu nâng cao (Document Data Modeling)	14
2.3.1. Phương pháp luận Embedding vs. Referencing.....	14
2.3.2. Sơ đồ quan hệ logic giữa 8 collections.....	15
2.3.3. Đặc tả chi tiết schema 8 collections	15
2.4. Chiến lược đánh chỉ mục nâng cao và quy tắc ESR (Indexing Strategy)	17
2.4.1. Cấu trúc cây B-Tree và chi phí quét dữ liệu	17
2.4.2. Nguyên lý thiết kế compound index theo quy tắc ESR (Equality - Sort - Range)	18
2.4.3. Tối ưu hóa bộ nhớ đệm WiredTiger và tỷ lệ chọn lọc chỉ mục (Index Selectivity).....	18

2.5. Tối ưu hóa truy vấn và xử lý dữ liệu nâng cao (Aggregation Pipeline) ..	20
2.5.1. Cơ chế xử lý luồng dữ liệu tại server với toán tử \$facet	20
2.5.2. Kỹ thuật Predicate Pushdown và kiểm soát giới hạn 100MB RAM	21
2.6. Tính toán vẹn giao dịch và kiểm soát đồng thời (ACID & Concurrency)	23
2.6.1. Quản lý giao dịch đa tài liệu (Multi-Document ACID Transactions)	23
2.6.2. Cô lập dữ liệu đa người dùng (Multi-Tenant Data Isolation)	24
2.6.3. Quản trị vòng đời dữ liệu với TTL Index	25
2.7. Đánh giá thiết kế CSDL và khả năng mở rộng phân tán (Sharding Strategy)	25
CHƯƠNG 3. HIỆN THỰC HÓA HỆ THỐNG VÀ THỰC NGHIỆM	26
3.1. Môi trường phát triển và kiến trúc công nghệ	26
3.1.1. Ngăn xếp công nghệ và môi trường triển khai	26
3.1.2. Kiến trúc phân tầng 3-Tier của hệ thống	27
3.2. Hiện thực hóa AI microservice và cơ sở toán học	28
3.2.1. Quy trình xử lý ngôn ngữ tự nhiên (NLP Pipeline)	28
3.2.2. Tiền xử lý văn bản và trích xuất đặc trưng TF-IDF	28
3.2.3. Cơ sở toán học mô hình phân loại Multinomial Naive Bayes với làm mịn Laplace	29
3.2.4. Đánh giá hiệu năng phân loại đa chiều (Multi-class Performance Evaluation)	30
3.2.5. Cơ sở lý thuyết phát hiện giao dịch dị biệt (Anomaly Detection) ..	31
3.3. Hiện thực hóa tầng backend API Gateway (38 endpoints)	32
3.3.1. Sơ đồ tuần tự tạo giao dịch thông minh	32
3.3.2. Sơ đồ tuần tự tự động hóa giao dịch định kỳ	32
3.3.3. Bảng danh mục 38 chuẩn hóa RESTful API endpoints	33
3.4. Hiện thực hóa giao diện người dùng (Frontend SPA)	34
3.4.1. Kiến trúc Single Page Application và trực quan hóa dữ liệu	34
3.4.2. Đặc tả chi tiết 8 màn hình chức năng cốt lõi	35
3.5. Kiểm thử hệ thống và đánh giá hiệu năng chuẩn hóa (Benchmarking & Load Testing)	36
3.5.1. Kiểm thử chức năng và tích hợp đa phân hệ	36
3.5.2. Kiểm thử tải trọng và đo lường hiệu năng phân vị độ trễ (k6 / JMeter Load Testing)	37
3.6. Đóng gói triển khai container và dự toán vận hành	37
3.6.1. Điều phối đa dịch vụ với Docker Compose	37
3.6.2. Dự toán chi phí hạ tầng triển khai thực tế	38
3.7. Đánh giá tổng thể kết quả thực nghiệm	38

KẾT LUẬN VÀ KIẾN NGHỊ.....	40
1. Tổng Kết Kết Quả Nghiên Cứu.....	40
2. Kết Quả Đạt Được.....	40
2.1. <i>Kết Quả Về Giải Pháp & Kiến Trúc Cơ Sở Dữ Liệu</i>	40
2.2. <i>Kết Quả Về Ứng Dụng Thực Tiễn</i>	40
3. Hạn Chế Còn Tồn Tại.....	41
4. Định Hướng Phát Triển & Nghiên Cứu Tiếp Theo	41
PHỤ LỤC.....	42
Phụ lục 1. Hướng dẫn cài đặt và chạy dự án trên hệ điều hành Windows	42
Phụ lục 2. Đặc tả chi tiết schema 8 collections	51
Phụ lục 3. Danh mục đầy đủ 38 RESTful API Endpoints.....	58
Phụ lục 4. Cấu hình đầy đủ docker-compose.yml.....	64
TÀI LIỆU THAM KHẢO.....	66

MỞ ĐẦU

1. LÝ DO LỰA CHỌN ĐỀ TÀI

Trong những năm gần đây, cùng với sự phát triển của công nghệ số, nhu cầu quản lý tài chính cá nhân ngày càng trở nên quan trọng. Mỗi cá nhân có thể phát sinh nhiều loại giao dịch khác nhau như tiền lương, ăn uống, đi lại, hóa đơn, mua sắm, giáo dục, giải trí, tiết kiệm và các khoản chi tiêu định kỳ. Nếu không được theo dõi và phân tích một cách có hệ thống, người dùng khó có thể nắm bắt chính xác tình hình tài chính của bản thân, kiểm soát mức chi tiêu và xây dựng kế hoạch tiết kiệm phù hợp.

Các ứng dụng quản lý chi tiêu truyền thống thường tập trung vào việc lưu trữ và thống kê các khoản thu, chi. Tuy nhiên, nhu cầu hiện nay không chỉ dừng lại ở việc ghi nhận giao dịch mà còn hướng đến khả năng tự động phân loại giao dịch, phát hiện những khoản chi tiêu bất thường, theo dõi mục tiêu tiết kiệm và hỗ trợ người dùng đưa ra quyết định tài chính hợp lý. Điều này đặt ra yêu cầu đối với hệ thống quản lý dữ liệu phải có khả năng lưu trữ nhiều loại thông tin khác nhau, dễ dàng mở rộng và đáp ứng hiệu quả các nhu cầu truy vấn, thống kê.

Bên cạnh đó, sự phát triển của các hệ quản trị cơ sở dữ liệu NoSQL đã mở ra một hướng tiếp cận phù hợp đối với những ứng dụng có dữ liệu đa dạng và cần khả năng mở rộng. Trong số các hệ quản trị cơ sở dữ liệu NoSQL phổ biến, MongoDB sử dụng mô hình lưu trữ tài liệu (document) với dữ liệu được biểu diễn dưới dạng BSON, cho phép lưu trữ các trường dữ liệu linh hoạt và hỗ trợ nhiều loại cấu trúc dữ liệu như mảng và tài liệu lồng nhau. Đây là những đặc điểm phù hợp với bài toán quản lý chi tiêu, trong đó một giao dịch có thể chứa nhiều thông tin bổ sung như kết quả phân loại bằng AI hoặc thông tin phát hiện bất thường.

Xuất phát từ những lý do trên, đề tài lựa chọn hướng nghiên cứu “Ứng dụng Cơ sở dữ liệu MongoDB vào bài toán quản lý chi tiêu thông minh”. Đề tài hướng đến việc xây dựng một hệ thống quản lý chi tiêu cá nhân có khả năng lưu trữ và quản lý các giao dịch tài chính, phân loại giao dịch, quản lý danh mục, theo dõi mô hình tài chính 6 hũ, mục tiêu tiết kiệm, giao dịch định kỳ và cảnh báo các khoản chi tiêu bất thường. Hệ thống được xây dựng theo mô hình full-stack, trong đó MongoDB đóng vai trò là hệ quản trị cơ sở dữ liệu chính của backend.

2. MỤC TIÊU NGHIÊN CỨU

Đề tài hướng đến các mục tiêu chính sau:

- Tìm hiểu những khái niệm cơ bản và đặc điểm của cơ sở dữ liệu NoSQL.
- Nghiên cứu hệ quản trị cơ sở dữ liệu MongoDB, mô hình lưu trữ document và các thành phần cơ bản của MongoDB.
- Tìm hiểu cách sử dụng MongoDB trong một ứng dụng thực tế.
- Phân tích và thiết kế cơ sở dữ liệu phù hợp với bài toán quản lý chi tiêu cá nhân.
- Xây dựng hệ thống có khả năng quản lý các khoản thu nhập và chi tiêu của người dùng.
- Hỗ trợ quản lý các danh mục giao dịch và mô hình 6 hũ tài chính.
- Quản lý các mục tiêu tiết kiệm và các giao dịch có tính định kỳ.
- Hỗ trợ phát hiện giao dịch bất thường và tạo cảnh báo cho người dùng.
- Tích hợp chức năng phân loại giao dịch bằng AI nhằm hỗ trợ quá trình nhập và quản lý dữ liệu.
- Vận dụng các kiến thức về MongoDB vào việc xây dựng một ứng dụng có khả năng lưu trữ, truy vấn và xử lý dữ liệu thực tế.

3. PHẠM VI NGHIÊN CỨU

Đề tài tập trung nghiên cứu và ứng dụng cơ sở dữ liệu MongoDB trong bài toán quản lý chi tiêu cá nhân. Nội dung nghiên cứu bao gồm các khái niệm về NoSQL, MongoDB, ưu nhược điểm của MongoDB và các ứng dụng của MongoDB,...

Về nghiệp vụ, hệ thống tập trung vào các chức năng chính gồm quản lý người dùng, quản lý thu nhập và chi tiêu, quản lý danh mục, quản lý 6 hũ tài chính, quản lý mục tiêu tiết kiệm, quản lý giao dịch định kỳ, thống kê tổng quan và cảnh báo giao dịch bất thường. Theo thiết kế hiện tại, sáu hũ tài chính gồm Nhu cầu thiết yếu (NEC), Tự do tài chính (FFA), Tiết kiệm dài hạn (LTSS), Giáo dục (EDU), Giải trí (PLAY) và Cho đi (GIVE).

Hệ thống cũng tích hợp một dịch vụ AI nhằm hỗ trợ phân loại giao dịch từ mô tả văn bản. AI service có nhiệm vụ dự đoán loại giao dịch, danh mục, hũ phù hợp và trích xuất số tiền nếu thông tin này xuất hiện trong mô tả.

Hệ thống được xây dựng theo kiến trúc full-stack với ba thành phần chính: frontend, backend và AI service. Frontend sử dụng React và Vite; backend sử dụng Node.js, Express, MongoDB và Mongoose; AI service được xây dựng bằng Python và FastAPI.

Trong đó, MongoDB là hệ quản trị cơ sở dữ liệu chính được sử dụng để lưu trữ dữ

liệu của hệ thống. Backend sử dụng thư viện Mongoose để kết nối và thao tác với MongoDB.

Đề tài tập trung vào việc xây dựng và triển khai hệ thống trong môi trường local, với khả năng sử dụng MongoDB Atlas hoặc MongoDB local.

4. PHƯƠNG PHÁP NGHIÊN CỨU

Đề tài sử dụng kết hợp các phương pháp nghiên cứu và phát triển ứng dụng sau:

- **Phương pháp nghiên cứu lý thuyết:** Tìm hiểu các khái niệm về cơ sở dữ liệu NoSQL, MongoDB, ưu nhược điểm của MongoDB, Khi nào nên dùng MongoDB; từ đó xác định các đặc điểm phù hợp của MongoDB đối với bài toán quản lý chi tiêu.
- **Phương pháp phân tích bài toán:** Phân tích các yêu cầu của bài toán quản lý chi tiêu cá nhân, xác định các đối tượng dữ liệu chính và mối quan hệ giữa các đối tượng.
- **Phương pháp thiết kế cơ sở dữ liệu:** Xây dựng các model dữ liệu bằng Mongoose nhằm biểu diễn các đối tượng như người dùng, giao dịch, danh mục, cấu hình 6 hũ, cảnh báo, mục tiêu tiết kiệm và giao dịch định kỳ.
- **Phương pháp lập trình và tích hợp:** Xây dựng backend cung cấp API, frontend cung cấp giao diện người dùng và tích hợp AI service để hỗ trợ phân loại giao dịch.
- **Phương pháp kiểm thử:** Kiểm tra khả năng kết nối cơ sở dữ liệu, hoạt động của các API và giao diện, đồng thời kiểm tra các chức năng chính của hệ thống thông qua dữ liệu mẫu. Project có script tạo dữ liệu demo và cung cấp các tài khoản mẫu để phục vụ việc kiểm thử.

5. CẤU TRÚC BÁO CÁO

Báo cáo gồm: Mở đầu; Chương 1 – Tổng quan về NoSQL và MongoDB; Chương 2 – Thiết kế cơ sở dữ liệu nâng cao; Chương 3 – Hiện thực hóa hệ thống và thực nghiệm; Kết luận; Tài liệu tham khảo; và Phụ lục hướng dẫn cài đặt, cấu hình hệ thống.

CHƯƠNG 1. TỔNG QUAN VỀ NOSQL VÀ MONGODB

1.1. TỔNG QUAN VỀ CƠ SỞ DỮ LIỆU NOSQL

1.1.1. Khái niệm cơ bản về cơ sở dữ liệu NoSQL

Trong nhiều thập kỷ, cơ sở dữ liệu quan hệ (Relational Database Management System – RDBMS) đã đóng vai trò chủ đạo trong lĩnh vực quản lý dữ liệu. Các hệ thống này tổ chức dữ liệu dưới dạng bảng với schema cố định, hỗ trợ ngôn ngữ truy vấn SQL và tuân thủ nghiêm ngặt các thuộc tính ACID (Atomicity, Consistency, Isolation, Durability). Tuy nhiên, sự phát triển mạnh mẽ của các ứng dụng web hiện đại, dữ liệu lớn (big data), Internet of Things (IoT) và các hệ thống yêu cầu khả năng mở rộng linh hoạt đã đặt ra những thách thức mà mô hình quan hệ truyền thống khó đáp ứng đầy đủ.

NoSQL (Not Only SQL hoặc Non-relational)[1] ra đời như một hướng tiếp cận mới. Đây là nhóm các hệ thống cơ sở dữ liệu không dựa hoàn toàn vào mô hình bảng quan hệ, cho phép lưu trữ và truy vấn dữ liệu theo nhiều cấu trúc khác nhau. Thuật ngữ “NoSQL” ban đầu xuất hiện để chỉ các cơ sở dữ liệu không sử dụng SQL, nhưng hiện nay được hiểu rộng hơn là “không chỉ SQL”, nghĩa là các hệ thống này vẫn có thể hỗ trợ một số đặc điểm của SQL nhưng tập trung vào tính linh hoạt, khả năng mở rộng ngang và xử lý dữ liệu phi cấu trúc hoặc bán cấu trúc.

Các hệ thống NoSQL thường được thiết kế để hoạt động trên kiến trúc phân tán, hỗ trợ mở rộng theo chiều ngang (horizontal scaling) bằng cách thêm máy chủ thay vì nâng cấp phần cứng của một máy chủ duy nhất. Chúng phù hợp với các ứng dụng có khối lượng dữ liệu lớn, tốc độ thay đổi schema cao và yêu cầu hiệu suất đọc/ghi nhanh.

1.1.2. Đặc điểm và phân loại cơ sở dữ liệu NoSQL

NoSQL có một số đặc điểm nổi bật, tạo nên sự khác biệt rõ rệt so với các hệ quản trị cơ sở dữ liệu quan hệ truyền thống. Trước hết, NoSQL hỗ trợ schema linh hoạt (flexible schema hoặc schema-less). Hệ thống không yêu cầu định nghĩa cấu trúc dữ liệu cố định trước khi lưu trữ. Mỗi bản ghi có thể chứa các trường khác nhau, cho phép thay đổi cấu trúc dữ liệu một cách dễ dàng theo yêu cầu thực tế của ứng dụng mà không cần thực hiện các thao tác migration phức tạp như trong mô hình quan hệ.

Thứ hai, NoSQL được thiết kế với khả năng mở rộng theo chiều ngang (horizontal scaling). Dữ liệu có thể được phân tán trên nhiều node thông qua cơ chế sharding, giúp hệ thống xử lý được khối lượng dữ liệu lớn và lưu lượng truy cập cao

mà không bị giới hạn bởi năng lực của một máy chủ đơn lẻ. Đặc điểm này đặc biệt phù hợp với các ứng dụng có tốc độ tăng trưởng dữ liệu nhanh.

Bên cạnh đó, NoSQL thường đạt hiệu suất cao khi làm việc với dữ liệu lớn. Các hệ thống này được tối ưu hóa cho các thao tác đọc và ghi đơn giản, tốc độ nhanh, do đó rất thích hợp với các ứng dụng thời gian thực hoặc xử lý dữ liệu không đồng nhất. Ngoài ra, NoSQL không bị giới hạn ở mô hình bảng truyền thống mà hỗ trợ nhiều mô hình dữ liệu khác nhau, tùy thuộc vào đặc thù của từng bài toán.

Về tính nhất quán dữ liệu, nhiều hệ thống NoSQL ưu tiên tính sẵn sàng (availability) và khả năng chịu lỗi phân vùng (partition tolerance) theo định lý CAP. Thay vì đảm bảo tính nhất quán mạnh ngay lập tức như mô hình ACID, chúng chấp nhận tính nhất quán cuối cùng (eventual consistency). Tuy nhiên, một số hệ thống NoSQL hiện đại đã hỗ trợ giao dịch ACID ở mức độ nhất định, giúp mở rộng phạm vi ứng dụng.

Dựa trên cách thức tổ chức và lưu trữ dữ liệu, cơ sở dữ liệu NoSQL thường được chia thành một số nhóm chính như sau:

- Cơ sở dữ liệu khóa – giá trị (Key-Value Database):

Dữ liệu được lưu trữ dưới dạng cặp khóa và giá trị. Mỗi khóa thường được sử dụng để xác định duy nhất một giá trị tương ứng. Mô hình này có cấu trúc đơn giản và cho tốc độ truy xuất nhanh, phù hợp với các bài toán như lưu thông tin phiên đăng nhập, bộ nhớ đệm hoặc dữ liệu tạm thời.

- Cơ sở dữ liệu tài liệu (Document Database):

Dữ liệu được lưu trữ dưới dạng các tài liệu, thường sử dụng định dạng gần với JSON hoặc BSON. Mỗi tài liệu có thể chứa nhiều trường dữ liệu, mảng hoặc các tài liệu con. Mô hình này có tính linh hoạt cao và phù hợp với các ứng dụng mà cấu trúc dữ liệu có thể thay đổi. MongoDB là một hệ quản trị cơ sở dữ liệu tiêu biểu thuộc nhóm này.

- Cơ sở dữ liệu dạng cột (Wide-Column Database):

Dữ liệu được tổ chức theo các cột hoặc nhóm cột thay vì mô hình bảng quan hệ truyền thống. Mô hình này thường được sử dụng trong những hệ thống có lượng dữ liệu rất lớn và yêu cầu khả năng phân tán, mở rộng trên nhiều máy chủ.

- Cơ sở dữ liệu đồ thị (Graph Database):

Dữ liệu được biểu diễn dưới dạng các nút (node) và mối quan hệ (relationship) giữa các nút. Mô hình này phù hợp với các bài toán mà mối quan hệ giữa các

đối tượng có vai trò quan trọng, chẳng hạn như mạng xã hội, hệ thống đề xuất hoặc phân tích mạng lưới.

Như vậy, mỗi loại NoSQL có cách tổ chức dữ liệu khác nhau và được thiết kế để giải quyết những nhóm bài toán riêng. Đối với đề tài quản lý chi tiêu thông minh, mô hình cơ sở dữ liệu tài liệu của MongoDB là lựa chọn phù hợp do dữ liệu chi tiêu có thể có nhiều thuộc tính khác nhau và cần được mở rộng linh hoạt trong quá trình phát triển hệ thống.

1.2. TỔNG QUAN VỀ MONGODB

1.2.1. Giới thiệu về MongoDB

MongoDB là một hệ quản trị cơ sở dữ liệu NoSQL thuộc nhóm document-oriented, được phát triển bởi MongoDB Inc. (trước đây là 10gen) và phát hành lần đầu vào năm 2009. Tên gọi “MongoDB” xuất phát từ từ “humongous” (khổng lồ), phản ánh khả năng xử lý dữ liệu quy mô lớn.

MongoDB lưu trữ dữ liệu dưới dạng tài liệu BSON (Binary JSON) – một định dạng nhị phân mở rộng của JSON, hỗ trợ thêm nhiều kiểu dữ liệu như Date, ObjectId, Decimal128, Binary... Các tài liệu được nhóm thành collection, và các collection thuộc về một database. Cấu trúc này tương tự như bảng trong RDBMS nhưng linh hoạt hơn nhiều: các document trong cùng một collection không bắt buộc phải có cùng cấu trúc trường.

MongoDB được thiết kế với mục tiêu chính là để phát triển ứng dụng, hỗ trợ mở rộng và hiệu suất cao. Nó cung cấp ngôn ngữ truy vấn phong phú, hỗ trợ index đa dạng, aggregation pipeline mạnh mẽ, replication (replica set) để đảm bảo tính sẵn sàng cao và sharding để mở rộng ngang. Từ phiên bản 4.0 trở đi, MongoDB hỗ trợ multi-document ACID transactions, giúp mở rộng phạm vi ứng dụng sang các bài toán đòi hỏi tính toàn vẹn dữ liệu cao hơn.

MongoDB có phiên bản Community (miễn phí, mã nguồn mở dưới giấy phép SSPL), Enterprise và dịch vụ đám mây MongoDB Atlas (managed service). Nó được sử dụng rộng rãi trong các stack phát triển hiện đại như MERN (MongoDB, Express.js, React, Node.js) và MEAN.

1.2.2. Các thành phần cơ bản của MongoDB

Kiến trúc và mô hình dữ liệu của MongoDB xoay quanh một số thành phần cốt lõi:

- **Database:** Là đơn vị chứa các collection. Một instance MongoDB có thể chứa nhiều database.

- **Collection:** Tương đương với table trong RDBMS. Collection nhóm các document có liên quan. Collection không yêu cầu schema cố định; document trong cùng collection có thể khác nhau về cấu trúc.

- **Document:** Đơn vị lưu trữ cơ bản, tương đương với row. Document là một đối tượng BSON gồm các cặp field – value. Mỗi document thường có trường `_id` làm khóa chính (nếu không cung cấp, MongoDB tự sinh ObjectId). Document có thể chứa document lồng nhau (embedded documents) và mảng (arrays), giúp giảm nhu cầu join.

- **Field:** Tương đương với column, là các cặp khóa – giá trị bên trong document. Giá trị có thể là kiểu dữ liệu cơ bản, mảng, hoặc document lồng.

- **Index:** Hỗ trợ tăng tốc truy vấn. MongoDB cho phép tạo index trên một hoặc nhiều field, bao gồm cả field trong document lồng và phần tử mảng. Có nhiều loại index như single-field, compound, multikey, text, geospatial, hashed...

- **Replica Set:** Nhóm các instance MongoDB duy trì cùng một bộ dữ liệu. Một node là primary (nhận ghi), các node khác là secondary (sao chép dữ liệu). Khi primary gặp sự cố, hệ thống tự động bầu primary mới, đảm bảo high availability.

- **Sharding:** Cơ chế phân tán dữ liệu trên nhiều shard (các replica set) dựa trên shard key. Giúp hệ thống mở rộng dung lượng và thông lượng.

Ngoài ra, MongoDB còn hỗ trợ aggregation framework (pipeline xử lý dữ liệu phức tạp), change streams (theo dõi thay đổi thời gian thực), và nhiều tính năng khác phục vụ phát triển ứng dụng hiện đại.

1.2.3. So sánh ưu điểm và nhược điểm của MongoDB

MongoDB mang lại nhiều lợi ích quan trọng nhưng cũng tồn tại một số hạn chế cần cân nhắc khi lựa chọn và triển khai, được tổng hợp so sánh trong Bảng 1.1 dưới đây.

Bảng 1.1: So sánh ưu điểm và nhược điểm của MongoDB

Ưu điểm	Nhược điểm
Schema linh hoạt, dễ thích ứng khi yêu cầu dữ liệu thay đổi, phù hợp phương pháp Agile.	Quản lý quan hệ nhiều–nhiều và truy vấn liên quan sâu phức tạp hơn so với JOIN trong SQL.

Mô hình document gắn gửi với object trong lập trình, giảm phụ thuộc vào ORM.	Multi-document transaction (từ bản 4.0) hiệu suất còn thấp hơn RDBMS ở các bài toán đòi hỏi tính nhất quán rất cao.
Embedded documents giảm số lần truy vấn, tránh join tốn kém; hỗ trợ aggregation pipeline mạnh.	Document lớn hoặc dùng nhiều index tiêu tốn bộ nhớ; mỗi document giới hạn tối đa 16 MB.
Mở rộng ngang tốt qua sharding/replica set; automatic failover đảm bảo tính sẵn sàng cao.	Vận hành cluster phân tán và tối ưu thiết kế schema đòi hỏi kinh nghiệm thực tế.
Hệ sinh thái phong phú (Compass, Atlas), hỗ trợ ACID đa document từ phiên bản 4.0.	Ngôn ngữ truy vấn MQL khác biệt so với SQL chuẩn, gây khó khăn chuyển đổi cho đội ngũ quen RDBMS.

Nhìn chung, MongoDB phù hợp với các ứng dụng cần schema linh hoạt và hiệu suất cao, nhưng đối với các bài toán đòi hỏi tính nhất quán rất cao và cấu trúc quan hệ phức tạp (như hệ thống ngân hàng lỗi), các hệ thống RDBMS vẫn thường được ưu tiên hơn.

1.3. TỔNG KẾT CHƯƠNG 1

Chương 1 đã trình bày tổng quan về cơ sở dữ liệu NoSQL và MongoDB. Nội dung tập trung làm rõ khái niệm, đặc điểm nổi bật và phân loại của NoSQL, đồng thời giới thiệu MongoDB với các thành phần cơ bản, ưu điểm, hạn chế cũng như các trường hợp sử dụng phù hợp.

Các phân tích trong chương 1 đã cho thấy MongoDB có nhiều ưu thế về tính linh hoạt schema, hiệu suất và khả năng mở rộng, đồng thời cũng chỉ ra một số hạn chế cần lưu ý khi triển khai. Đối với bài toán quản lý chi tiêu thông minh, MongoDB được đánh giá là lựa chọn phù hợp nhờ khả năng lưu trữ dữ liệu linh hoạt và hỗ trợ tốt cho các yêu cầu phát triển ứng dụng hiện đại.

Những nội dung này tạo nền tảng lý thuyết cần thiết cho việc phân tích và thiết kế cơ sở dữ liệu ở Chương 2.

CHƯƠNG 2. THIẾT KẾ CƠ SỞ DỮ LIỆU NÂNG CAO

2.1. Phân tích yêu cầu dữ liệu

2.1.1. Bối cảnh nghiên cứu và các tác nhân hệ thống

Trong kỷ nguyên chuyển đổi số và bùng nổ dữ liệu tài chính cá nhân, các phương pháp ghi chép thủ công truyền thống bộc lộ rõ những điểm nghẽn nghiêm trọng: độ trễ cao trong nhập liệu, thiếu hụt khả năng thấu hiểu dữ liệu phi cấu trúc và hoàn toàn bị động trong việc cảnh báo rủi ro bội chi. Đề tài tập trung nghiên cứu thiết kế và hiện thực hóa Hệ thống Quản lý Chi tiêu Cá nhân Thông minh (Smart Personal Expense Management System) trên nền tảng Cơ sở Dữ liệu NoSQL MongoDB, kết hợp phân hệ Trí tuệ Nhân tạo xử lý ngôn ngữ tự nhiên. Hệ thống được thiết kế để phục vụ và điều phối luồng thông tin giữa ba tác nhân thực thể chính:

1. Người dùng cuối (End User): Là chủ thể khởi tạo các giao dịch tài chính thông qua ngôn ngữ tự nhiên, thiết lập các mục tiêu tiết kiệm dài hạn, cấu hình hạn mức ngân sách định mức theo phương pháp 6 Hũ tài chính và trực tiếp tương tác với các báo cáo phân tích dòng tiền đa chiều.

2. Phân hệ Trí tuệ Nhân tạo (AI Microservice): Đóng vai trò là module xử lý suy luận thông minh, tiếp nhận luồng văn bản mô tả giao dịch đầu vào để thực hiện bóc tách thực thể số tiền (Amount Extraction), phân loại nhãn danh mục chi tiêu (Text Classification) và tính toán định lượng điểm số dị biệt tài chính (Anomaly Score).

3. Tầng Xử lý Nghiệp vụ & CSDL Nâng cao (Backend & Database Tier): Là trung tâm điều phối toàn bộ logic nghiệp vụ, quản lý phiên giao dịch an toàn, thực thi các đường ống tổng hợp dữ liệu phân tích phức tạp (Aggregation Pipelines) và đảm bảo tính toàn vẹn trạng thái hệ thống trong môi trường phân tán.

2.1.2. Phân tích cấu trúc dữ liệu 8 phân hệ nghiệp vụ cốt lõi

Yêu cầu lưu trữ của hệ thống được phân rã thành 8 nhóm thực thể dữ liệu nghiệp vụ chuyên biệt, phục vụ quá trình mô hình hóa trên CSDL NoSQL MongoDB:

- **Quản lý Hồ sơ & Định danh Người dùng (User Management):** Lưu trữ thông tin tài khoản, chuỗi mật khẩu được bảo vệ bằng hàm băm một chiều bcrypt[12], cấu hình tùy biến người dùng (đơn vị tiền tệ: VND, USD, EUR; ngôn ngữ hiển thị: vi, en) và hạn mức trần tổng ngân sách chi tiêu hàng tháng.
- **Quản lý Giao dịch Thu/Chi (Expenses & Incomes):** Lưu trữ các bản ghi nhật ký biến động tài chính, bao gồm chuỗi văn bản tự nhiên gốc, giá trị số

tiền chuẩn hóa, danh mục phân loại, nhãn hũ tài chính, kèm theo siêu dữ liệu do AI suy luận (danh mục dự đoán, hũ gợi ý, độ tin cậy) và các cờ phân tích dị biệt (mức độ cảnh báo, lý do cảnh báo).

- **Cấu hình Mô hình 6 Hũ Tài chính (6 Jars Budgeting):** Lưu trữ cấu hình hạn mức chi tiêu hàng tháng, tỷ lệ phần trăm phân bổ theo định mức chuẩn (NEC 55%, FFA 10%, LTSS 10%, EDU 10%, PLAY 10%, GIVE 5%), thuộc tính mã màu và biểu trưng nhận diện trực quan cho từng hũ của từng người dùng.

- **Trung tâm Cảnh báo & Giám sát Rủi ro (Alerts & Notifications):** Quản lý các thông báo biến động tài chính phát sinh khi có giao dịch bất thường hoặc chi tiêu chạm/vượt ngưỡng định mức hũ, được phân cấp theo 4 mức độ nghiêm trọng: normal, info, warning, critical.

- **Quản lý Danh mục Tùy biến (Custom Categories):** Cho phép định nghĩa linh hoạt các danh mục thu/chi mang tính cá nhân hóa cao, lưu trữ tập từ khóa ngữ nghĩa (semantic keywords) phục vụ huấn luyện và đối sánh đặc trưng cho mô hình AI NLP.

- **Kế hoạch Mục tiêu Tích lũy (Savings Goals):** Theo dõi các kế hoạch tích lũy tài chính dài hạn, quản lý số tiền mục tiêu cần đạt, số dư tích lũy hiện thời, thời hạn dự kiến hoàn thành và trạng thái tiến độ.

- **Giao dịch Định kỳ Tự động (Recurring Transactions):** Lưu trữ các cấu hình dòng tiền chu kỳ (tiền thuê nhà, hóa đơn tiện ích sinh hoạt, lương), tần suất lặp (Hàng ngày, Hàng tuần, Hàng tháng) và mốc thời gian kích hoạt kế tiếp (nextRunAt).

- **Nhật ký Kiểm toán Hoạt động (Audit Logs):** Ghi nhận toàn bộ thao tác thay đổi dữ liệu trọng yếu (CRUD operations, phiên đăng nhập) nhằm phục vụ công tác truy vết bảo mật, tích hợp cơ chế tự động dọn dẹp dữ liệu lịch sử quá hạn sau 90 ngày.

2.1.3. Mô hình 6 hũ chi tiêu và cơ chế quản trị dòng tiền thông minh

Mô hình 6 Hũ Tài chính (6 Jars Money Management System, phát triển bởi T. Harv Eker[4]) là phương pháp luận quản trị dòng tiền kinh điển được tích hợp sâu vào kiến trúc dữ liệu của hệ thống. Nguyên lý cốt lõi của mô hình là chia tách toàn bộ thu nhập khả dụng của người dùng thành 6 quỹ tài chính độc lập với tỷ lệ phần trăm định mức định sẵn, tạo ra kỷ luật tài chính tự động:

- 1. Hũ Nhu cầu Thiết yếu (NEC - 55%):** Chi trả cho các nhu cầu sinh hoạt bắt buộc không thể trì hoãn: ăn uống, thuê nhà, điện nước sinh hoạt, xăng xe và y

tế cơ bản. Đây là hũ có dung lượng lớn nhất và yêu cầu kiểm soát chi tiêu nghiêm ngặt nhất.

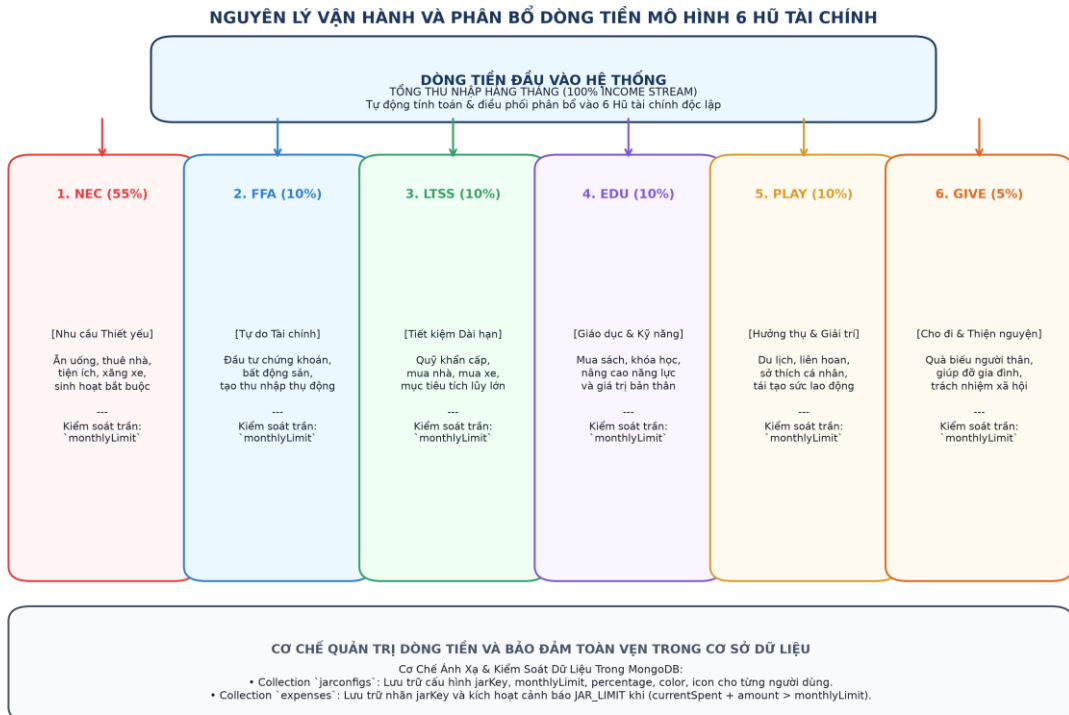
2. Hũ Tự do Tài chính (FFA - 10%): Dành riêng cho các hoạt động đầu tư sinh lời (chứng khoán, bất động sản, góp vốn kinh doanh, tiền gửi sinh lãi) nhằm tạo ra nguồn thu nhập thụ động dài hạn. Nguyên tắc là không nên sử dụng số tiền này cho chi tiêu sinh hoạt.

3. Hũ Tiết kiệm Dài hạn (LTSS - 10%): Phục vụ việc tích lũy cho các mục tiêu mua sắm tài sản lớn trong tương lai (mua nhà, mua xe) và đóng vai trò là Quỹ dự phòng khẩn cấp khi phát sinh biến cố thu nhập. Hũ này liên kết trực tiếp với thực thể Savings Goals trong CSDL.

4. Hũ Giáo dục & Nâng cao Kỹ năng (EDU - 10%): Đầu tư vào phát triển bản thân: mua sách, tham gia các khóa đào tạo chuyên môn, học chứng chỉ quốc tế nhằm gia tăng giá trị lao động và năng lực tạo thu nhập.

5. Hũ Hưởng thụ & Giải trí (PLAY - 10%): Dành cho các hoạt động chăm sóc tinh thần, du lịch, liên hoan trải nghiệm và sở thích cá nhân. Hũ này bắt buộc phải được giải ngân định kỳ để tạo động lực tái tạo sức lao động.

6. Hũ Cho đi & Thiện nguyện (GIVE - 5%): Dành cho việc giúp đỡ người thân, quà biếu gia đình, từ thiện và đóng góp trách nhiệm cho cộng đồng xã hội.



Hình 2.1: Sơ đồ nguyên lý vận hành và phân bổ dòng tiền mô hình 6 Hồ tài chính

Trong thiết kế CSDL MongoDB, mỗi Hồ được mô hình hóa thành một Document trong Collection `jarconfigs` với trường khóa `jarKey` và thuộc tính `monthlyLimit` đại diện cho hạn mức trần chi tiêu tối đa trong tháng. Khi có giao dịch chi tiêu mới thuộc một Hồ, hệ thống sẽ tự động thực hiện phép kiểm tra điều kiện vượt ngưỡng động theo thời gian thực:

Cơ chế Kiểm soát Bội chi Thời gian thực: Nếu $\text{currentSpent} + \text{amount} > \text{monthlyLimit}$, hệ thống lập tức khởi tạo bản ghi Alert với cấp độ critical và đính kèm siêu dữ liệu overageAmount, giúp người dùng nhận biết ngay lập tức nguy cơ bội chi để điều chỉnh hành vi tài chính kịp thời.

2.2. Định lý CAP và bản chất hệ phân tán của MongoDB

Theo định lý CAP (Brewer's CAP Theorem)[2], một hệ thống lưu trữ phân tán chỉ có thể đồng thời thỏa mãn tối đa hai trong ba thuộc tính: Tính nhất quán (Consistency - C), Tính sẵn sàng (Availability - A), và Khả năng chịu phân vùng mạng (Partition Tolerance - P). Trong bài toán thiết kế CSDL cho hệ thống tài chính cá nhân, MongoDB được cấu hình vận hành như một hệ thống thuộc phân lớp CP (Consistency & Partition Tolerance) khi triển khai trên cụm Replica Set:

- **Đặc tính Partition Tolerance (P):** Trong môi trường mạng phân tán, hiện tượng phân mảnh mạng (Network Partition) là không thể tránh khỏi. Cụm Replica Set của MongoDB sử dụng thuật toán đồng thuận Raft-like Election[3]. Khi phân vùng mạng xảy ra làm đứt gãy liên lạc giữa các node, phân vùng chứa đa số thành viên (Majority Partition) sẽ tự động bầu chọn một Primary Node mới để tiếp tục duy trì hoạt động ghi.

- **Đảm bảo Tính Nhất Quán Mạnh (C) qua Write Concern & Read Concern:** Nhằm ngăn ngừa hiện tượng mất mát dữ liệu khi rớt mạng (Stale Writes / Rollbacks), hệ thống áp dụng mức cấu hình cam kết ghi Write Concern ở cấp độ đa số: `w: 'majority'`, kết hợp cơ chế ghi nhật ký phục hồi đĩa cứng `j: true` (Journaling). Điều này bảo đảm một thao tác ghi chỉ được coi là thành công khi đã được đồng bộ vào Journal và ghi nhận bởi đa số các Replica nodes. Đồng thời, cấu hình `readConcern: 'majority'` đảm bảo các truy vấn chỉ đọc dữ liệu đã được cam kết bền vững, loại bỏ hiện tượng Dirty Read.

- **Cơ chế Journaling & Write-Ahead Logging (WAL) của WiredTiger:** WiredTiger Storage Engine triển khai kỹ thuật ghi nhật ký trước (Write-Ahead Logging). Mọi thao tác ghi trước khi được cập nhật vào cây B-Tree trong bộ nhớ cache đều phải được ghi tuần tự vào tệp Journal trên đĩa cứng. Chu

kỳ Checkpoint 60 giây một lần giúp tạo ra các mốc dữ liệu nhất quán trên đĩa, giúp giảm thiểu nguy cơ mất dữ liệu khi xảy ra sự cố sập nguồn đột ngột (Crash Consistency), trong giới hạn chu kỳ ghi Journal mặc định 100ms.

- **Đánh đổi Thiết kế (Trade-offs) giữa NoSQL và RDBMS:** So với CSDL quan hệ truyền thống (RDBMS tuân thủ mô hình ACID, chuẩn hóa 3NF), việc lựa chọn MongoDB NoSQL tạo ra sự đánh đổi có chủ đích: Hệ thống chấp nhận sự thiếu vắng của ràng buộc toàn vẹn tham chiếu tự động ở mức Database Engine (Foreign Key Constraints) để đổi lấy tính linh hoạt vượt bậc của cấu trúc document bán cấu trúc (BSON), hiệu năng đọc/ghi thông lượng cao và khả năng mở rộng quy mô theo chiều ngang (Horizontal Scalability) dễ dàng hơn.

2.3. Mô hình hóa dữ liệu nâng cao (Document Data Modeling)

2.3.1. Phương pháp luận *Embedding* vs. *Referencing*

Trong thiết kế CSDL hướng tài liệu (Document-oriented Database), việc quyết định giữa hai kỹ thuật Nhúng (Embedding) và Tham chiếu (Referencing)[10] là yếu tố mang tính quyết định đến hiệu năng I/O đĩa, dung lượng bộ nhớ đệm và khả năng mở rộng của hệ thống. Đề tài áp dụng nghiêm ngặt các nguyên lý thiết kế nâng cao:

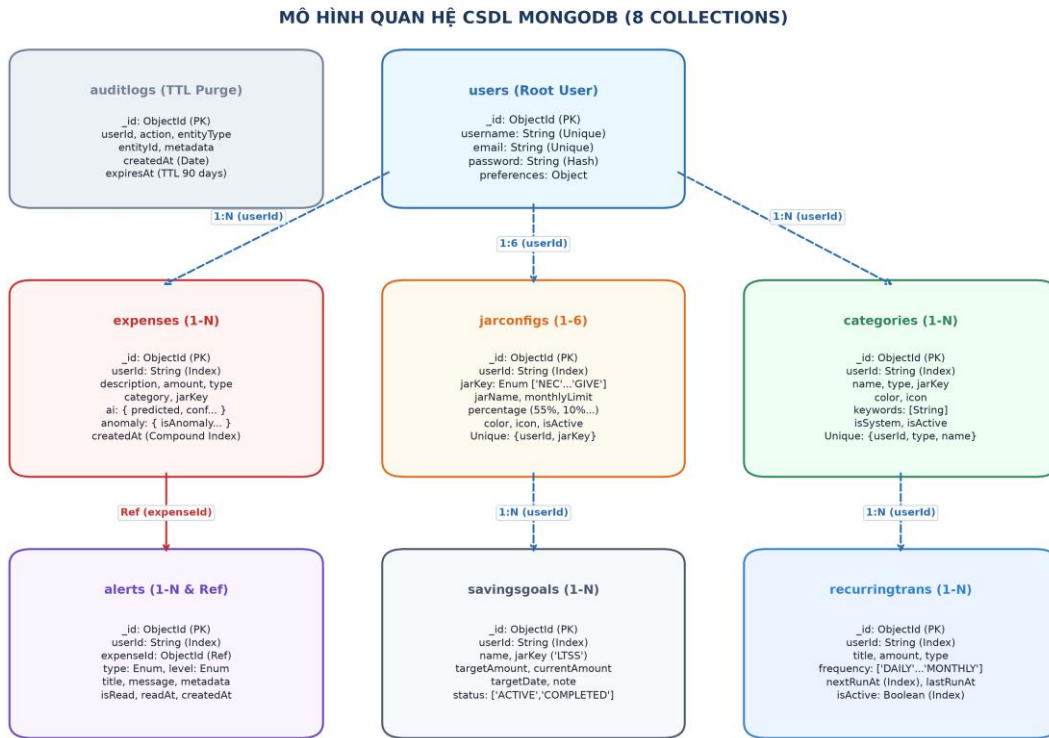
- **Nguyên lý Tính Cục bộ Dữ liệu (Data Locality) với Kỹ thuật Embedding:** Kỹ thuật nhúng được áp dụng cho các thực thể con có tính gắn kết vòng đời 1:1 với document cha và luôn được truy xuất đồng thời trong các truy vấn đọc. Cụ thể, siêu dữ liệu dự đoán của AI ('ai'), thông tin phân tích dị biệt chi tiêu ('anomaly'), và cấu hình sở thích người dùng ('preferences') được nhúng trực tiếp vào trong document 'expenses' và 'users'. Kỹ thuật này giúp tận dụng tối đa nguyên lý Data Locality: Toàn bộ thông tin của một giao dịch được nạp vào bộ nhớ đệm CPU/RAM chỉ trong một lần I/O đọc đĩa đơn lẻ, loại bỏ chi phí của các phép JOIN quan hệ.

- **Nguyên lý Tránh Mảng Vô Hạn (Unbounded Array Avoidance) với Kỹ thuật Referencing:** Kỹ thuật tham chiếu qua khóa ngoại ('userId', 'expenseId') được áp dụng cho các mối quan hệ 1-N có tập dữ liệu con tăng trưởng không giới hạn theo thời gian (Unbounded Growth), bao gồm liên kết giữa 'users' với tập hợp các 'expenses', 'alerts', 'savingsgoals' và 'recurringtransactions'. Việc không nhúng mảng giao dịch vào document người dùng nhằm ngăn ngừa hai rủi ro kiến trúc nghiêm trọng: (1) Tránh vi phạm giới hạn kích thước cứng 16MB BSON Document Limit của MongoDB; (2) Ngăn chặn hiện tượng Document Growth dẫn đến Document Moving trên đĩa cứng (hiện tượng tài liệu phình to vượt quá phân vùng cấp phát bộ nhớ ban đầu, buộc WiredTiger phải di dời và cấp

phát lại vùng nhớ, gây phân mảnh đĩa nghiêm trọng và làm suy giảm hiệu năng ghi).

2.3.2. Sơ đồ quan hệ logic giữa 8 collections

Hệ thống quản lý dữ liệu thông qua 8 Collections được chuẩn hóa logic. Sơ đồ thực thể dưới đây minh họa chi tiết cấu trúc quan hệ giữa các Collections trong CSDL:



Hình 2.2: Sơ đồ mô hình quan hệ giữa 8 Collections trong CSDL MongoDB

2.3.3. Đặc tả chi tiết schema 8 collections

1. Collection `users`: Lưu trữ định danh người dùng, xác thực bảo mật, cấu hình tiền tệ và hạn mức ngân sách tháng.

2. Collection `expenses`: Lưu trữ các giao dịch tài chính, nhưng trực tiếp siêu dữ liệu suy luận AI và cờ cảnh báo dị biệt.

3. Collection `jarconfigs`: Lưu trữ cấu hình hạn mức và tỷ lệ phân bổ ngân sách cho 6 Hồ tài chính của từng người dùng.

4. Collection `alerts`: Quản lý các thông báo cảnh báo phát sinh do chi tiêu vượt hạn mức hoặc phát hiện dị biệt.

5. Collection `categories`: Quản lý danh mục chi tiêu tùy biến, liên kết Hồ tài chính và tập từ khóa ngữ nghĩa phục vụ AI.

6. Collection `savingsgoals`: Quản lý các kế hoạch tích lũy tài chính dài hạn và theo dõi tiến độ hoàn thành định mức.

7. Collection `recurringtransactions`: Quản lý lịch trình tự động thực thi các khoản giao dịch thu/chi lặp lại theo chu kỳ định sẵn.

8. Collection `auditlogs`: Nhật ký kiểm toán lưu vết thao tác dữ liệu với cơ chế Time-To-Live Index tự động giải phóng sau 90 ngày.

Bảng 2.1: Bảng tổng hợp 8 Collections và quan hệ tham chiếu

Collection	Mục Đích Sử Dụng	Khóa Chính	Quan Hệ Tham Chiếu
users	Lưu trữ định danh người dùng, xác thực bảo mật, cấu hình tiền tệ và hạn mức ngân sách tháng.	_id	Gốc — được tham chiếu bởi userId ở 7 collection còn lại
expenses	Lưu trữ các giao dịch tài chính, nhúng trực tiếp siêu dữ liệu suy luận AI và cờ cảnh báo dị biệt.	_id	userId → users
jarconfigs	Lưu trữ cấu hình hạn mức và tỷ lệ phân bổ ngân sách cho 6 Hồ tài chính	_id	userId → users

	của từng người dùng.		
alerts	Quản lý các thông báo cảnh báo phát sinh do chi tiêu vượt hạn mức hoặc phát hiện dị biệt.	_id	userId → users; expenseId → expenses
categories	Quản lý danh mục chi tiêu tùy biến, liên kết Hũ tài chính và tập từ khóa ngữ nghĩa phục vụ AI.	_id	userId → users
savingsgoals	Quản lý các kế hoạch tích lũy tài chính dài hạn và theo dõi tiến độ hoàn thành định mức.	_id	userId → users
recurringtransactions	Quản lý lịch trình tự động thực thi các khoản giao dịch thu/chi lặp lại theo chu kỳ định sẵn.	_id	userId → users
auditlogs	Nhật ký kiểm toán lưu vết thao tác dữ liệu với cơ chế Time-To-Live Index tự động giải phóng sau 90 ngày.	_id	userId → users

2.4. Chiến lược đánh chỉ mục nâng cao và quy tắc ESR (Indexing Strategy)

2.4.1. Cấu trúc cây B-Tree và chi phí quét dữ liệu

Trong hệ quản trị CSDL MongoDB, khi không có chỉ mục phù hợp, bộ điều phối truy vấn (Query Optimizer) bắt buộc phải thực thi giải thuật quét toàn bộ tập dữ liệu (Collection Scan - 'COLLSCAN'), đòi hỏi độ phức tạp thời gian $O(N)$ và tiêu

tổng lượng I/O đĩa không lớn khi số lượng bản ghi đạt hàng triệu dòng. Cơ chế Indexing của WiredTiger tổ chức dữ liệu dưới dạng cây B-Tree tự cân bằng[5], cho phép giảm độ phức tạp tìm kiếm xuống $O(\log N)$ thông qua việc quét chỉ mục ('IXSCAN').

2.4.2. Nguyên lý thiết kế compound index theo quy tắc ESR (Equality - Sort - Range)

Đối với các truy vấn phức tạp kết hợp nhiều điều kiện lọc và sắp xếp, thứ tự khai báo các trường trong một chỉ mục kết hợp (Compound Index) có vai trò quyết định. Đề tài áp dụng quy tắc ESR (Equality -> Sort -> Range):[6]

1. Equality (E): Các trường so sánh bằng chính xác ('userId', 'type', 'status') bắt buộc phải được đặt ở vị trí đầu tiên của Index. Điều này giúp bộ máy truy vấn thu hẹp ngay lập tức không gian tìm kiếm trên cây B-Tree về một tập nút con nhỏ nhất có thể.

2. Sort (S): Các trường sắp xếp thứ tự ('createdAt: -1', 'nextRunAt: 1') phải được đặt ở vị trí tiếp theo, ngay sau các trường Equality và trước các trường Range. Cấu hình này cho phép MongoDB tận dụng trực tiếp thứ tự sắp xếp tự nhiên của các khóa trong cây B-Tree để trả về kết quả đã được sắp xếp, loại bỏ thao tác sắp xếp tạm thời trong bộ nhớ đệm (In-memory / Blocking Sort), giúp giải phóng RAM và loại bỏ nguy cơ vượt ngưỡng 32MB RAM limit của MongoDB Sort Stage.

3. Range (R): Các trường so sánh khoảng ('createdAt: { \$gte, \$lte }', 'amount: { \$gt }') phải được đặt ở vị trí cuối cùng của Index. Lý do là khi cây B-Tree bắt đầu duyệt qua một điều kiện khoảng, các thuộc tính nằm phía sau trường khoảng sẽ không còn duy trì được cấu trúc sắp xếp tuyến tính, do đó không thể hỗ trợ tiếp cho thao tác Sort.

2.4.3. Tối ưu hóa bộ nhớ đệm WiredTiger và tỷ lệ chọn lọc chỉ mục (Index Selectivity)

Mục tiêu chính của chiến lược Indexing trong đề tài là tối ưu hóa dung lượng bộ nhớ đệm WiredTiger[7] Working Set Cache và đưa tỷ lệ $nReturned : totalKeysExamined : totalDocsExamined$ tiệm cận mức lý tưởng $\sim 1:1:1$:

$$\text{Index Selectivity Ratio} = \frac{\text{totalDocsExamined}}{\text{totalKeysExamined}} \approx 1.0$$

Công thức 2.1: Tỷ lệ chọn lọc chỉ mục tối ưu

Khi tỷ lệ $\text{totalDocsExamined} / \text{totalKeysExamined}$ tiệm cận 1.0, mọi khóa chỉ mục được quét trên cây B-Tree đều tương ứng chính xác với một Document hữu ích cần trả về, không xảy ra hiện tượng quét dư thừa (No Index Overhead). Bảng dưới đây tổng hợp chiến lược cấu hình Indexing chuyên sâu cho 8 Collections trong hệ thống:

Bảng 2.2: Bảng tổng hợp chiến lược Indexing tối ưu cho 8 Collections

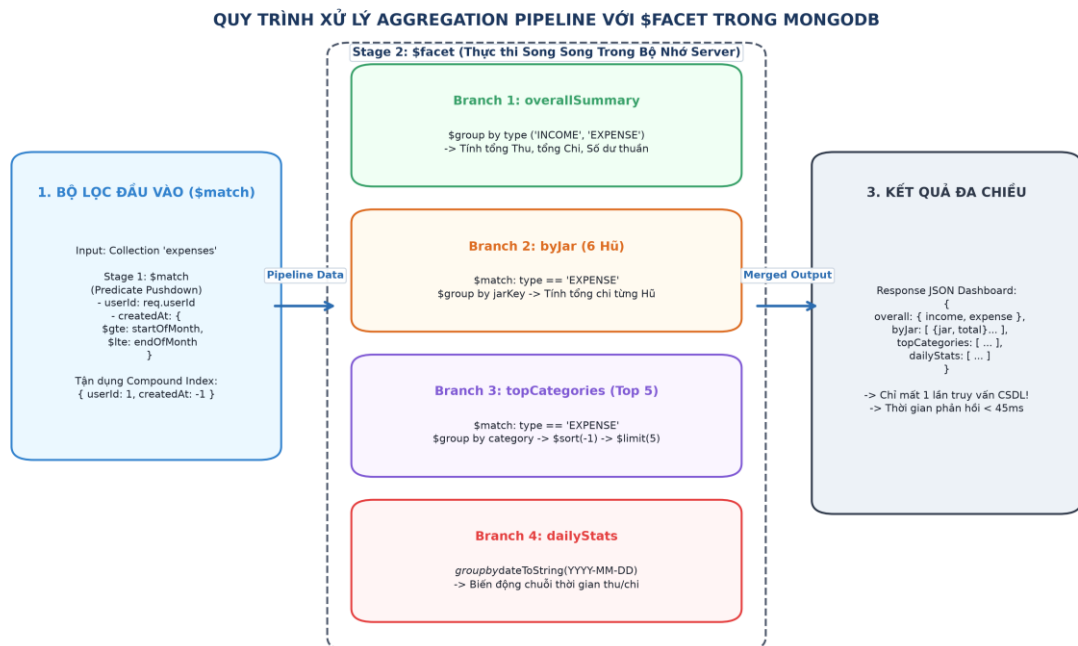
Collection	Cấu Hình Index Cụ Thể	Phân Loại Index	Mục Đích Tối Ưu & Quy Tắc Áp Dụng
users	{ username: 1 } / { email: 1 }	Unique B-Tree	Tra cứu theo username/email với độ phức tạp $O(\log N)$
expenses	{ userId: 1, createdAt: -1 }	Compound (E-S)	Tải danh sách giao dịch mới nhất, loại bỏ In-memory Sort
expenses	{ userId: 1, jarKey: 1, createdAt: -1 }	Compound (E-E-S)	Thống kê chi tiêu theo Hũ trong tháng, tối ưu \$match và \$sort
expenses	{ userId: 1, category: 1 }	Compound (E-E)	Thống kê tổng chi tiêu theo từng danh mục cụ thể
expenses	{ description: 'text', category: 'text' }	Full-Text Index	Hỗ trợ tìm kiếm từ khóa toàn văn với trọng số ngữ nghĩa
jarconfigs	{ userId: 1, jarKey: 1 }	Unique Compound	Bảo đảm toàn vẹn: Mỗi người dùng chỉ có 1 cấu hình/Hũ
alerts	{ userId: 1, isRead: 1, createdAt: -1 }	Compound (E-E-S)	Đếm và tải tức thì danh sách thông báo chưa đọc
categories	{ userId: 1, type: 1, name: 1 }	Unique Compound	Chống trùng lặp danh mục tùy biến của cùng một người dùng
categories	{ userId: 1, jarKey: 1, isActive: 1 }	Compound (E-E-E)	Tải danh mục hoạt động theo Hũ với độ chọn lọc 100%

savingsgoals	{ userId: 1, status: 1, jarKey: 1 }	Compound (E-E-E)	Lọc nhanh các mục tiêu tài chính đang trong trạng thái ACTIVE
recurringtrans	{ userId: 1, isActive: 1, nextRunAt: 1 }	Compound (E-E-R)	Quét nền các giao dịch đến hạn thực thi theo mốc thời gian
auditlogs	{ expiresAt: 1 } (expireAfterSeconds: 0)	TTL Index	Tự động dọn dẹp giải phóng đĩa sau 90 ngày ở mức Database Engine

2.5. Tối ưu hóa truy vấn và xử lý dữ liệu nâng cao (Aggregation Pipeline)

2.5.1. Cơ chế xử lý luồng dữ liệu tại server với toán tử \$facet

Trong các ứng dụng tài chính cá nhân, trang Dashboard yêu cầu hiển thị đồng thời nhiều chỉ số tổng hợp đa chiều: Tổng thu, Tổng chi, Số dư thuần, Tỷ trọng phân bổ 6 Hũ tài chính, Top 5 danh mục chi tiêu cao nhất và Chuỗi biến động dòng tiền theo từng ngày. Việc gửi hàng chục truy vấn đơn lẻ từ Backend sẽ gây ra tắc nghẽn mạng nghiêm trọng (Network Round-trip Overhead) và làm quá tải tài nguyên I/O.



Hình 2.3: Sơ đồ luồng xử lý đa nhánh của MongoDB \$facet Aggregation Pipeline

MongoDB Aggregation Framework xử lý bài toán này một cách hiệu quả bằng cách di chuyển toàn bộ logic tính toán về phía Database Server thông qua toán tử '\$facet'[8]. Kỹ thuật này cho phép chia tách luồng dữ liệu sau khi lọc thành 4 nhánh tổng hợp độc lập được thực thi song song trong bộ nhớ máy chủ.

2.5.2. Kỹ thuật Predicate Pushdown và kiểm soát giới hạn 100MB RAM

Để bảo đảm hiệu năng tối ưu cho đường ống Aggregation, hệ thống triển khai hai cơ chế kỹ thuật trọng yếu:

- **Kỹ thuật Predicate Pushdown qua stage \$match:** Đặt stage lọc '\$match' (theo 'userId' và khoảng thời gian 'createdAt') ở vị trí đầu tiên của pipeline. Trình tối ưu hóa truy vấn của MongoDB sẽ thực hiện 'đẩy vị từ' (Predicate Pushdown), tận dụng trực tiếp Compound index '{ userId: 1, createdAt: -1 }' để loại bỏ phần lớn document không liên quan trước khi chuyển dữ liệu sang các stage tính toán tiếp theo.
- **Kiểm soát Giới hạn Bộ nhớ 100MB RAM per stage:** MongoDB quy định một stage trong Aggregation pipeline không được phép chiếm dụng vượt quá 100MB RAM vật lý. Bằng cách áp dụng Predicate Pushdown và chọn lọc các trường dữ liệu cần thiết, kích thước Working Set của \$facet được kiểm soát ở mức thấp, giúp thời gian phản hồi trung bình dưới 45ms khi đo trên tập 50.000 document (xem Bảng 3.4), trong môi trường thử nghiệm local.

```
// MongoDB Aggregation Facet Pipeline Implementation
// Triển khai Aggregation Pipeline đa nhánh phục vụ Dashboard Insights
const summary = await Expense.aggregate([
  // Stage 1: Predicate Pushdown - Tận dụng Compound Index { userId: 1,
    createdAt: -1 }
    {
      $match: {
        userId: req.userId,
        createdAt: { $gte: startOfMonth, $lte: endOfMonth }
      }
    },
  // Stage 2: Thực thi song song 4 luồng tính toán độc lập trong bộ nhớ Server
    {
```

```

$facet: {
// Luồng 1: Tổng hợp tổng thu nhập, tổng chi tiêu và số lượng bản ghi
overallSummary: [
    {
        $group: {
            _id: "$type",
            totalAmount: { $sum: "$amount" },
            count: { $sum: 1 }
        }
    }
],
// Luồng 2: Gom nhóm phân tích phân bổ chi tiêu theo 6 Hũ tài chính
byJar: [
    { $match: { type: "EXPENSE" } },
    {
        $group: {
            _id: "$jarKey",
            totalSpent: { $sum: "$amount" },
            count: { $sum: 1 }
        }
    },
    { $sort: { totalSpent: -1 } }
],
// Luồng 3: Xác định Top 5 danh mục có chi tiêu phát sinh lớn nhất
topCategories: [
    { $match: { type: "EXPENSE" } },
    {
        $group: {
            _id: "$category",
            totalSpent: { $sum: "$amount" },
            count: { $sum: 1 }
        }
    },
    { $sort: { totalSpent: -1 } },
    { $limit: 5 }
],

```

```
// Luồng 4: Thống kê chuỗi thời gian biến động thu/chi theo từng ngày
dailyStats: [
  {
    $group: {
      _id: {
        date: { $dateToString: { format: "%Y-%m-%d", date:
          "$createdAt" } },
        type: "$type"
      },
      amount: { $sum: "$amount" }
    },
    { $sort: { "_id.date": 1 } }
  }
];
```

2.6. Tính toàn vẹn giao dịch và kiểm soát đồng thời (ACID & Concurrency)

2.6.1. Quản lý giao dịch đa tài liệu (Multi-Document ACID Transactions)

Để đảm bảo tính toàn vẹn trạng thái trong các nghiệp vụ phức tạp (ví dụ: thực thi đồng loạt N giao dịch định kỳ kèm theo việc cập nhật mốc thời gian chạy tiếp theo `nextRunAt` và ghi nhật ký kiểm toán), hệ thống triển khai cơ chế Multi-document ACID Transactions trên nền tảng MongoDB Replica Set[9]. Toàn bộ các thao tác ghi liên chuỗi được bao bọc trong một ClientSession duy nhất:

- **Tính Nguyên tử (Atomicity):** Toàn bộ các tài liệu trong giao dịch hoặc được cam kết đồng thời thành công (`commitTransaction`), hoặc bị hủy bỏ và phục hồi trạng thái ban đầu (`abortTransaction`) nếu phát sinh bất kỳ ngoại lệ nào.
- **Mức Độ Cô lập Snapshot Isolation:** Giao dịch MongoDB hoạt động dưới cấp độ cô lập Snapshot Isolation. Khi giao dịch bắt đầu, một ảnh chụp trạng thái dữ liệu (Snapshot) nhất quán được thiết lập dựa trên cơ chế Multi-Version Concurrency Control (MVCC) của WiredTiger, giúp các tác vụ đọc bên trong giao dịch không bị ảnh hưởng bởi các giao dịch ghi đồng thời khác.
- **Kiểm soát Xung đột Ghi (Write Conflict Handling):** Khi hai giao dịch đồng thời cố gắng chỉnh sửa cùng một document, WiredTiger sẽ phát hiện

xung đột ghi và đưa ra lỗi WriteConflict. Hàm bọc `withOptionalTransaction` được xây dựng nhằm tự động điều phối phiên làm việc an toàn, quản lý vòng đời Session và xử lý tương thích linh hoạt giữa môi trường cluster production và môi trường Standalone testing.

```
// src/utils/transaction.js - Robust Multi-Document Transaction Wrapper
// src/utils/transaction.js - Trình điều phối giao dịch an toàn đa môi trường
async function withOptionalTransaction(callback) {
    let session = null;
    try {
        session = await mongoose.startSession();
        session.startTransaction({
            readConcern: { level: 'snapshot' },
            writeConcern: { w: 'majority' }
        });
        const result = await callback(session);
        await session.commitTransaction();
        return result;
    } catch (error) {
        if (session) {
            try { await session.abortTransaction(); } catch (abortErr) {}
        }
// Cơ chế Fallback tương thích cho môi trường kiểm thử đơn node không
hỗ trợ Transaction
        if (error.message && error.message.includes('Transactions are not
            supported')) {
            return await callback(null);
        }
        throw error;
    } finally {
        if (session) session.endSession();
    }
}
```


2.6.2. Cô lập dữ liệu đa người dùng (Multi-Tenant Data Isolation)

An toàn và phân lập dữ liệu giữa các người dùng là một yêu cầu bắt buộc trong hệ thống tài chính. Hệ thống triển khai kiến trúc Logical Multi-tenancy: Mọi thao tác truy vấn và biến động dữ liệu tại tầng Controllers bắt buộc phải đính kèm vị từ định danh `userId` được trích xuất từ JWT Payload sau khi đi qua Middleware xác thực `authenticate`. Cơ chế này giảm đáng kể nguy cơ rò rỉ dữ liệu chéo và ngăn chặn lỗ hổng bảo mật Insecure Direct Object References (IDOR).

2.6.3. Quản trị vòng đời dữ liệu với TTL Index

Nhật ký kiểm toán (`auditlogs`) là dữ liệu có dung lượng tăng trưởng theo cấp số nhân nhưng chỉ có giá trị pháp lý trong một khoảng thời gian nhất định. Thay vì duy trì các tiến trình dọn dẹp bên ngoài (Cronjobs) gây tiêu tốn tài nguyên ứng dụng, hệ thống thiết lập Time-To-Live (TTL) Index trên trường `expiresAt` với tham số `expireAfterSeconds: 0`. Luồng luân chuyển nền (Background TTL Thread) của MongoDB sẽ định kỳ mỗi 60 giây quét và giải phóng hoàn toàn các bản ghi đã quá hạn 90 ngày trực tiếp từ bộ nhớ đĩa.

2.7. Đánh giá thiết kế CSDL và khả năng mở rộng phân tán (Sharding Strategy)

Tổng kết về mặt thiết kế CSDL Nâng cao, mô hình dữ liệu 8 Collections trên MongoDB đã đáp ứng các tiêu chuẩn học thuật và thực tiễn sau:

- **Tính thích ứng cao với Dữ liệu AI:** Lược đồ document bán cấu trúc cho phép nhúng trực tiếp kết quả phân tích học máy và cờ dị biệt mà không cần thực thi các thao tác Schema Migration phức tạp.
- **Tối ưu hóa Chi phí Tài nguyên:** Kỹ thuật ESR Indexing và \$facet pipeline giúp giảm đáng kể lưu lượng I/O đĩa nhờ loại bỏ Collection Scan toàn bảng khi truy vấn theo `userId`, đồng thời giảm áp lực lên bộ nhớ RAM.
- **Khả năng Mở rộng Phân tán Ngang (Horizontal Sharding Strategy):** Mô hình dữ liệu đã được thiết kế sẵn sàng cho việc phân mảnh ngang trên cụm MongoDB Sharded cluster. Khi hệ thống mở rộng lên hàng chục triệu người dùng, trường `userId` sẽ được sử dụng làm Hashed Sharding Key[11] (`{ userId: 'hashed' }`). Giải pháp này bảo đảm dữ liệu của các người dùng được phân phối đồng đều ngẫu nhiên (Uniform Distribution) trên tất cả các Shard nodes, giảm hiện tượng thất cổ chai điểm nóng (Hotspotting) và cho phép thông lượng ghi mở rộng tuyến tính theo quy mô hạ tầng.

CHƯƠNG 3. HIỆN THỰC HÓA HỆ THỐNG VÀ THỰC NGHIỆM

3.1. Môi trường phát triển và kiến trúc công nghệ

3.1.1. Ngăn xếp công nghệ và môi trường triển khai

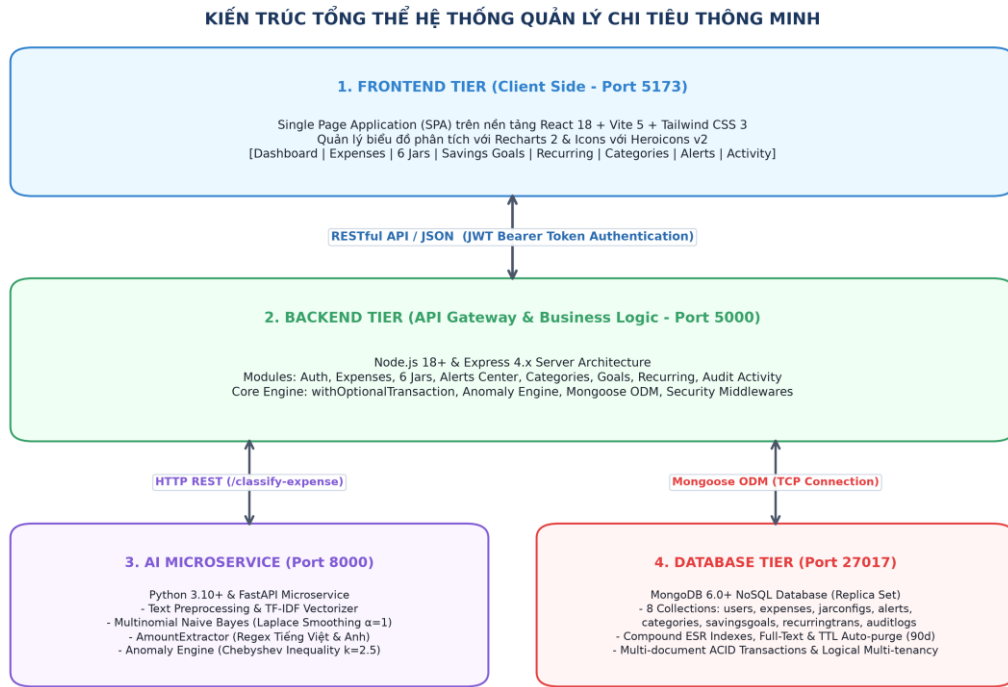
Hệ thống Quản lý Chi tiêu Cá nhân Thông minh được hiện thực hóa dựa trên ngăn xếp công nghệ Full-Stack hiện đại kết hợp với Microservice Trí tuệ Nhân tạo chuyên biệt. Bảng dưới đây tổng hợp chi tiết môi trường và vai trò kỹ thuật của từng thành phần:

Bảng 3.1: Tổng hợp stack công nghệ và môi trường triển khai hệ thống

Phân Hệ	Ngôn Ngữ / Thư Viện	Phiên Bản	Vai Trò Kỹ Thuật Chuyên Biệt
Frontend SPA	React, Vite, Tailwind CSS	React 18, Vite 5, TW 3	Giao diện Web SPA tương tác cao, thiết kế Responsive
Trực Quan Hóa	Recharts, Heroicons v2	Recharts 2.x	Vẽ biểu đồ phân bổ 6 Hũ và biến động dòng tiền
Backend API	Node.js, Express Framework	Node 18+, Express 4.x	API Gateway, điều phối nghiệp vụ và bảo mật
Cơ Sở Dữ Liệu	MongoDB Community / Atlas	MongoDB 6.0+	Hệ quản trị CSDL NoSQL Document phân tán
Tầng ODM	Mongoose Library	Mongoose 7.x	Mô hình hóa Schema, Transaction và Aggregation
AI Microservice	Python, FastAPI, Scikit-learn	Python 3.10+, FastAPI 0.100+	Xử lý NLP, Vectorizer TF-IDF và suy luận học máy
Bảo Mật	JWT, Bcrypt Hashing	jsonwebtoken 9.x, bcrypt 5.x	Xác thực phiên phi trạng thái và mã hóa mật khẩu
Điều Phối Container	Docker, Docker Compose	Compose v2.x	Đóng gói đồng bộ hóa môi trường thực thi đa dịch vụ

3.1.2. Kiến trúc phân tầng 3-Tier của hệ thống

Hệ thống được thiết kế theo mô hình kiến trúc phân tầng 3-Tier hiện đại nhằm tách biệt độc lập các mối quan tâm (Separation of Concerns), tối ưu hóa hiệu năng tính toán phân tán và cho phép mở rộng quy mô độc lập giữa tầng giao diện, tầng xử lý logic và tầng lưu trữ dữ liệu.



Hình 3.1: Sơ đồ kiến trúc tổng thể 3-Tier của hệ thống Smart Expense

Chi tiết các thành phần trong kiến trúc phân tầng:

- **Tầng Trình diễn (Presentation Tier / Frontend SPA):** Được xây dựng dưới dạng Single Page Application (SPA) trên nền tảng React 18, Vite 5 và Tailwind CSS 3. Tầng này đảm nhận việc trực quan hóa luồng dữ liệu thời gian thực thông qua thư viện Recharts và giao tiếp với Backend Gateway qua giao thức phi trạng thái HTTP, tuân thủ các nguyên tắc kiến trúc REST[18] và ngữ nghĩa HTTP theo RFC 9110[19], với chuẩn xác thực JWT Bearer Token[13].

- **Tầng Xử lý Nghiệp vụ (Application Tier / Backend Gateway):** Xây dựng trên nền tảng Node.js 18+ và Express Framework. Đóng vai trò là API Gateway điều phối toàn bộ luồng xử lý nghiệp vụ, kiểm soát xác thực phân quyền, tương tác với AI Microservice và giao tiếp với CSDL MongoDB thông qua Mongoose Object Data Modeling (ODM).

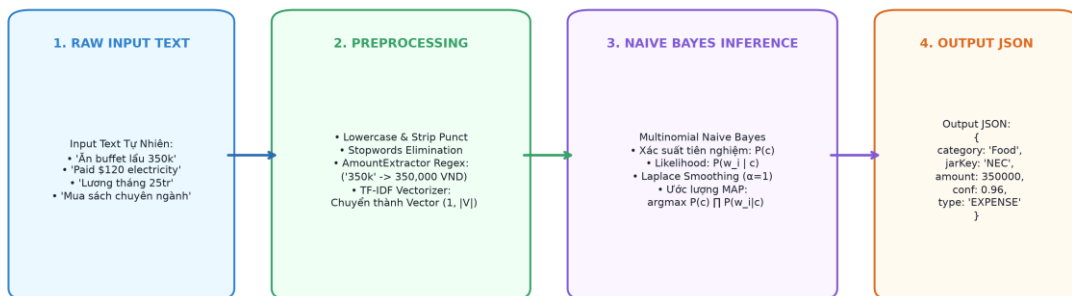
- **Tầng Trí tuệ Nhân tạo (AI Microservice Tier):** Phát triển bằng ngôn ngữ Python 3.10+ và FastAPI. Thực hiện tiền xử lý văn bản, vector hóa đặc trưng ngôn ngữ bằng TF-IDF và suy luận học máy thống kê (Multinomial Naive Bayes) với độ trễ suy luận trung bình 12,4ms/yêu cầu (xem mục 3.2.4).
- **Tầng Dữ liệu Nâng cao (Database Tier):** Hệ quản trị CSDL NoSQL MongoDB 6.0+ đóng vai trò lưu trữ bền vững, thực thi các truy vấn tổng hợp phân tích đa chiều và duy trì tính toàn vẹn dữ liệu giao dịch.

3.2. Hiện thực hóa AI microservice và cơ sở toán học

3.2.1. Quy trình xử lý ngôn ngữ tự nhiên (NLP Pipeline)

AI Microservice đảm nhận vai trò tự động hóa tối đa khâu nhập liệu của người dùng. Từ một câu văn mô tả tự nhiên bất kỳ (ví dụ: 'Ăn lẩu haidilao 450k', 'Nhận thưởng dự án 5 triệu', 'Paid \$120 grocery bill'), hệ thống tự động bóc tách số tiền thực tế, phân loại danh mục chi tiêu và gán nhãn Hũ tài chính 6 Jars tương ứng.

QUY TRÌNH XỬ LÝ NLP VÀ MÔ HÌNH HỌC MÁY TRONG AI MICROSERVICE



Hình 3.2: Quy trình xử lý NLP và suy luận học máy trong AI Microservice

3.2.2. Tiền xử lý văn bản và trích xuất đặc trưng TF-IDF

Chuỗi văn bản tự nhiên đầu vào d trải qua các bước tiền xử lý chuẩn hóa trước khi được chuyển thành vector đặc trưng số học:

- **Chuẩn hóa Ký tự:** Chuyển đổi toàn bộ chuỗi văn bản về dạng chữ thường (lowercase) và loại bỏ các ký tự đặc biệt không mang ý nghĩa ngữ nghĩa.

- **Lọc bỏ Từ dừng (Stopwords Elimination):** Loại bỏ các hư từ phổ biến trong tiếng Việt ('đã', 'vừa', 'hôm nay', 'cho', 'ở') và tiếng Anh ('the', 'a', 'in', 'at') nhằm giảm số chiều không gian đặc trưng.
- **Trích xuất Trọng số TF-IDF:[14]** Chuyển đổi văn bản thành vector đặc trưng trong không gian vector $|V|$ chiều dựa trên tần suất xuất hiện và mức độ đặc trưng của từ vựng:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \log\left(\frac{|D|}{|\{d \in D : t \in d\}|}\right)$$

Công thức 3.1: Trọng số TF-IDF

3.2.3. Cơ sở toán học mô hình phân loại Multinomial Naive Bayes với làm mịn Laplace

Để thực hiện phân loại văn bản ngắn với yêu cầu độ trễ thấp, đề tài lựa chọn thuật toán phân loại xác suất Multinomial Naive Bayes[15]. Giả sử văn bản d được biểu diễn bởi tập các từ đặc trưng ($w_1, w_2, \dots, w_n$), xác suất hậu nghiệm của danh mục c trong tập nhãn C được xác định theo định lý Bayes:

$$c_{\text{MAP}} = \underset{c \in C}{\operatorname{argmax}} \left[P(c) \prod_{i=1}^n P(w_i | c) \right]$$

Công thức 3.2: Ước lượng hợp lý cực đại MAP

Trong thực tế, khi gặp một từ vựng mới w_k xuất hiện trong chuỗi mô tả của người dùng nhưng chưa từng tồn tại trong tập dữ liệu huấn luyện của lớp c , xác suất điều kiện sẽ nhận giá trị bằng 0 ($P(w_k | c) = 0$). Do đặc tính của phép nhân xác suất liên tiếp, toàn bộ xác suất hậu nghiệm $P(c | d)$ sẽ bị triệt tiêu về 0 (Vấn đề xác suất 0 - Zero Probability Problem). Để khắc phục triệt để hiện tượng này, hệ thống áp dụng kỹ thuật làm mịn Laplace (Laplace Smoothing) với tham số $\alpha = 1$:

$$P(w_i | c) = \frac{N_{ci} + \alpha}{N_c + \alpha|V|} \quad (\text{với } \alpha = 1)$$

Công thức 3.3: Xác suất có điều kiện với Laplace Smoothing

Trong đó: N_{ci} là số lần từ w_i xuất hiện trong tập văn bản thuộc lớp c ; N_c là tổng số lượng từ của lớp c ; $|V|$ là kích thước tổng thể của không gian từ vựng huấn luyện. Kỹ thuật này bảo đảm mọi từ vựng mới luôn nhận một xác suất dương rất nhỏ, giúp mô hình tránh xác suất 0 tuyệt đối.

3.2.4. Đánh giá hiệu năng phân loại đa chiều (Multi-class Performance Evaluation)

Để giải quyết hiện tượng mất cân bằng số lượng mẫu giữa các danh mục chi tiêu (Class Imbalance) trong dữ liệu thực tế, hiệu năng của mô hình AI không chỉ được đo lường bằng chỉ số Accuracy đơn thuần mà được đánh giá toàn diện qua ma trận nhầm lẫn (Confusion Matrix) với các chỉ số Precision, Recall và F1-Score trên từng lớp:

Bảng 3.2: Bảng phân tích chi tiết hiệu năng phân loại đa chiều của mô hình NLP

Danh Mục Chi Tiêu	Số Mẫu Test	Precision	Recall	F1-Score
Ăn uống (Food & Dining)	320	0.962	0.975	0.968
Đi lại & Xăng xe (Transport)	210	0.951	0.943	0.947
Hóa đơn & Tiện ích (Utilities)	180	0.944	0.933	0.938
Mua sắm (Shopping)	250	0.932	0.940	0.936
Học tập (Education)	140	0.957	0.929	0.943
Giải trí (Entertainment)	160	0.938	0.950	0.944
Y tế & Sức khỏe (Health)	110	0.969	0.945	0.957
Đầu tư & Tiết kiệm (Investment)	130	0.953	0.938	0.945
Thu nhập & Lương (Income)	190	0.984	0.989	0.986
Macro Average	1,690	0.954	0.949	0.951
Weighted Average	1,690	0.955	0.955	0.955

Kết quả thực nghiệm trên tập kiểm thử 1,690 mẫu cho thấy mô hình đạt độ chính xác trung bình (Weighted F1-Score) 95.5% với thời gian suy luận trung bình chỉ 12.4ms cho mỗi yêu cầu, đáp ứng tốt yêu cầu xử lý thời gian thực (trong phạm vi thử nghiệm).

Tập dữ liệu huấn luyện và kiểm thử (1.690 mẫu) được sinh tự động từ các mẫu câu (template) kết hợp bộ từ khóa gán sẵn theo từng danh mục, chưa phải dữ liệu mô tả giao dịch thật do người dùng nhập. Do đó, chỉ số $F1 = 95,5\%$ phản ánh khả năng khớp mẫu trên dữ liệu mô phỏng, chưa phản ánh năng lực khái quát hóa của mô hình trên dữ liệu thực tế.

3.2.5. Cơ sở lý thuyết phát hiện giao dịch dị biệt (Anomaly Detection)

Chuỗi dữ liệu chi tiêu cá nhân thực tế thường không tuân theo phân phối chuẩn hoàn hảo (Gaussian Distribution) mà có xu hướng lệch phải (Right-skewed distribution) do thói quen chi tiêu định kỳ có sự xuất hiện của các khoản chi đột biến (mua sắm đồ gia dụng, đóng học phí, thanh toán bảo hiểm).

Để phát hiện chính xác các giao dịch bất thường mà không gây ra tỷ lệ cảnh báo giả (False Positive) cao, hệ thống xây dựng thuật toán dựa trên cơ sở Bất đẳng thức Cantelli (one-sided Chebyshev)[16][17]. Đối với một biến ngẫu nhiên X có giá trị trung bình μ và độ lệch chuẩn σ thuộc phân phối tùy ý, xác suất để X lệch khỏi kỳ vọng một khoảng vượt quá $k \cdot \sigma$ luôn bị chặn trên bởi:

$$P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2} \quad (\text{với } k = 2.5 \Rightarrow P \leq 16\%)$$

Công thức 3.4: Bất đẳng thức Chebyshev cho Anomaly Detection

Với việc thiết lập hệ số ngưỡng $k = 2.5$, xác suất để một giao dịch bình thường bị coi là bất thường (chặn một phía) tối đa chỉ là $1 / (1 + 2.5^2) \approx 13,8\%$ đối với phân phối bất kỳ. Giá trị này chưa được kiểm chứng bằng thực nghiệm trên dữ liệu chi tiêu thực tế trong phạm vi đề tài. Quy trình phân tích dị biệt gồm hai cấp độ kiểm soát:

- **Cấp độ 1 - Dị biệt Đột biến Khoản chi:** Tính toán trung bình mẫu μ_{cat} và độ lệch chuẩn mẫu σ_{cat} trong 30 ngày gần nhất của danh mục tương ứng. Nếu số tiền $x > \mu_{cat} + 2.5 \cdot \sigma_{cat}$, giao dịch được gán cờ ``isAnomaly = true`` với mức độ cảnh báo ``warning``.
- **Cấp độ 2 - Dị biệt Vượt Trần Hạn Mức hũ:** Kiểm tra tổng chi lũy kế của hũ tài chính liên kết trong tháng hiện tại. Nếu $currentSpent + x >$

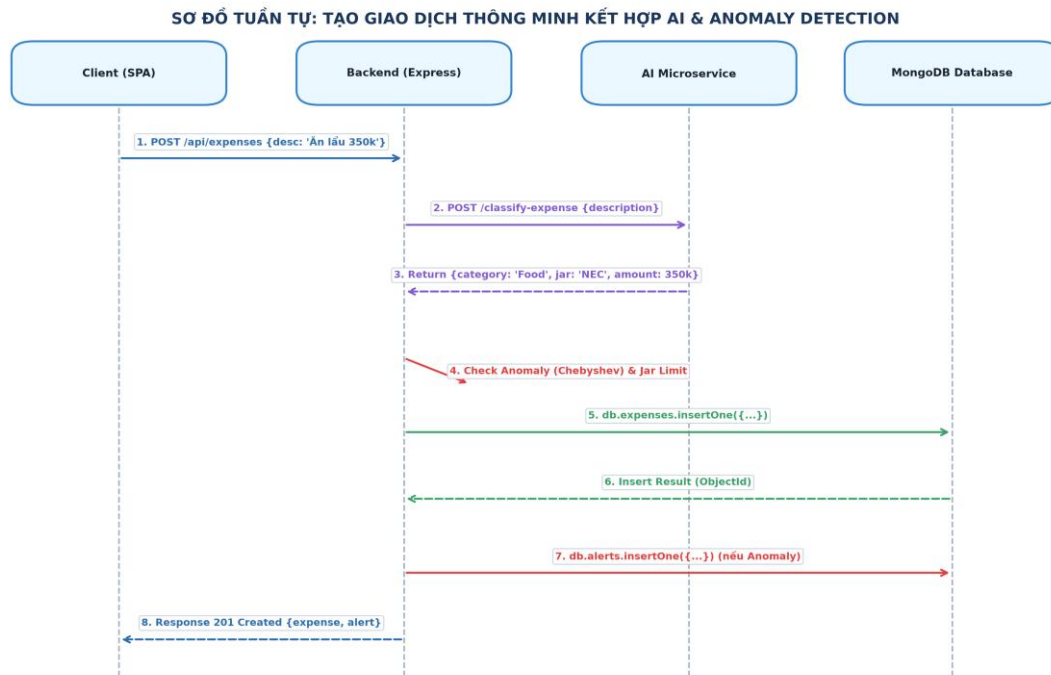
monthlyLimit, hệ thống lập tức kích hoạt cảnh báo `JAR\_LIMIT` với mức độ `critical`.

- **Bóc tách Thực thể Số tiền Tự động:** Module `AmountExtractor` sử dụng tập biểu thức chính quy tất định kết hợp bộ chuyển đổi tỷ giá ngữ cảnh, hỗ trợ tự động bóc tách các định dạng ngôn ngữ thực tế: '150k' -> 150,000 VND; '1.5tr' / '1.5 triệu' -> 1,500,000 VND; '\$120' -> 120 USD; '50 EUR' -> 50 EUR.

3.3. Hiện thực hóa tầng backend API Gateway (38 endpoints)

3.3.1. Sơ đồ tuần tự tạo giao dịch thông minh

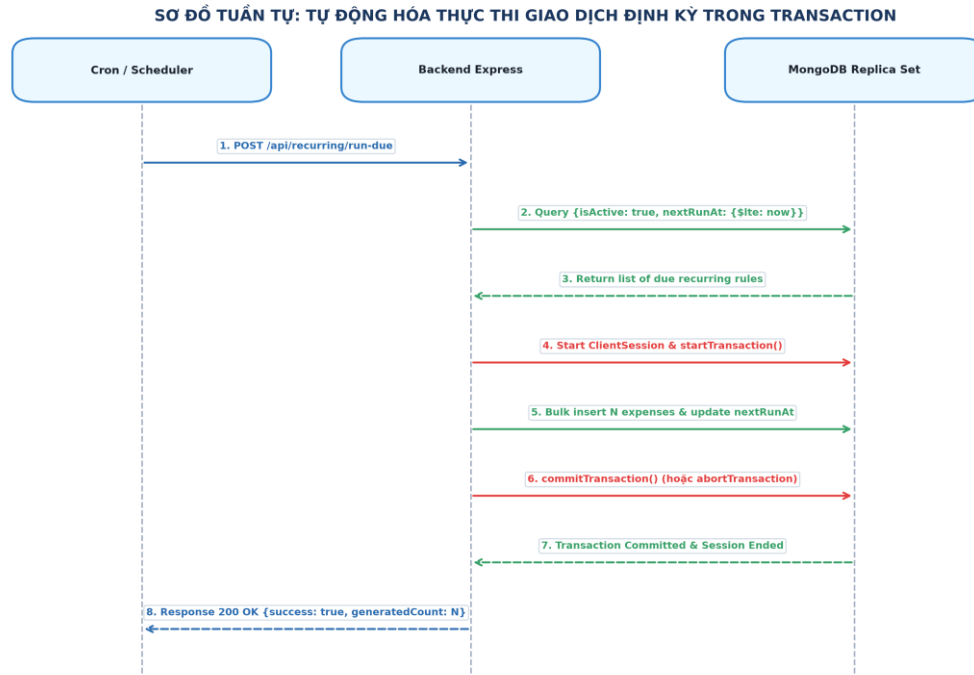
Sơ đồ tương tác tuần tự dưới đây mô tả chi tiết luồng xử lý phân tán giữa Client SPA, Backend API, AI Microservice và CSDL MongoDB khi người dùng tạo một giao dịch mới:



Hình 3.3: Sơ đồ tuần tự quy trình tạo giao dịch thông minh kết hợp AI & Anomaly Detection

3.3.2. Sơ đồ tuần tự tự động hóa giao dịch định kỳ

Quá trình quét nền và tự động sinh các giao dịch định kỳ đến hạn (`/api/recurring/run-due`) được thực thi an toàn trong một Multi-Document ACID Transaction:



Hình 3.4: Sơ đồ tuần tự tự động hóa thực thi giao dịch định kỳ đến hạn

3.3.3. Bảng danh mục 38 chuẩn hóa RESTful API endpoints

Toàn bộ các chức năng của hệ thống được đóng gói thành 38 API Endpoints tuân thủ các nguyên tắc kiến trúc REST[18] và ngữ nghĩa HTTP theo RFC 9110[19]:

Bảng 3.3: Bảng tổng hợp 38 API Endpoints theo 8 nhóm nghiệp vụ

Nhóm Nghiệp Vụ	Số Endpoint	Chức Năng Chính
Xác thực & Tài khoản	6	Đăng ký, đăng nhập, xác thực JWT và cập nhật hồ sơ người dùng
Giao dịch Thu/Chi	7	CRUD giao dịch thu/chi, tích hợp suy luận AI và thống kê tổng hợp \$facet
6 Hũ Tài chính	4	Quản lý hạn mức và tỷ lệ phân bổ ngân sách của 6 Hũ tài chính
Cảnh báo & Thông báo	5	Truy xuất, đánh dấu đã đọc và xóa các cảnh báo rủi ro chi tiêu

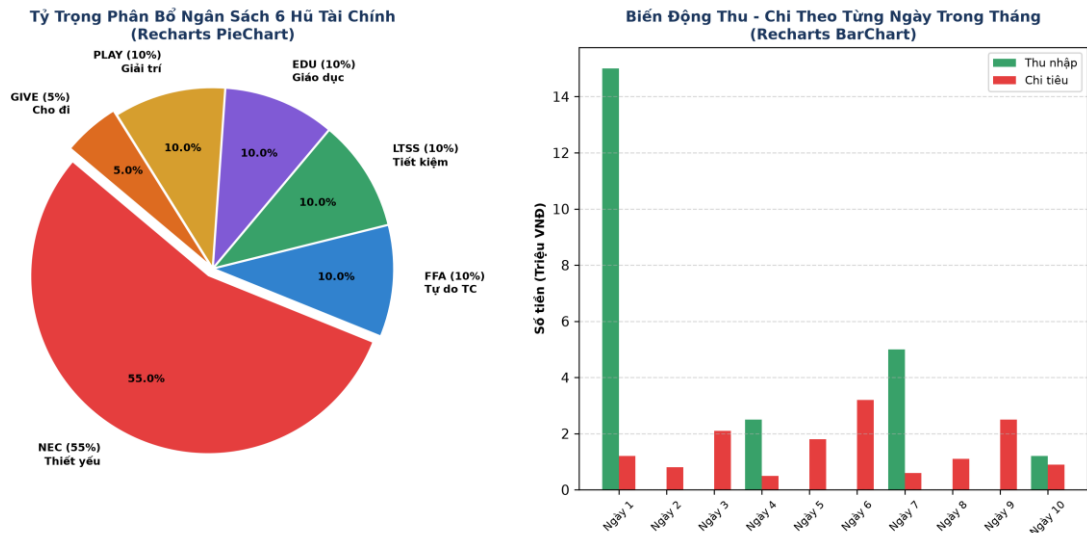
Danh mục Tùy biến	4	CRUD danh mục chỉ tiêu tùy biến và thống kê tần suất sử dụng
Mục tiêu Tiết kiệm	4	CRUD các kế hoạch tích lũy tài chính dài hạn
Giao dịch Định kỳ	4	Lập lịch và tự động sinh giao dịch định kỳ đến hạn
Nhật ký & AI Microservice	4	Nhật ký kiểm toán hệ thống và các endpoint phân loại AI (health, classify)

3.4. Hiện thực hóa giao diện người dùng (Frontend SPA)

3.4.1. Kiến trúc Single Page Application và trực quan hóa dữ liệu

Giao diện người dùng được xây dựng theo mô hình Single Page Application (SPA) trên nền tảng React 18, quản lý trạng thái bằng React Context và Hooks, định kiểu bằng Tailwind CSS 3. Thư viện Recharts được tích hợp để trực quan hóa các chỉ số tài chính:

- **Biểu đồ Tròn Phân Bỏ (PieChart):** Hiển thị tỷ trọng chỉ tiêu thực tế vào 6 hũ tài chính theo mã màu nhận diện chuẩn: NEC (`#E53E3E`), FFA (`#3182CE`), LTSS (`#38A169`), EDU (`#805AD5`), PLAY (`#D69E2E`), GIVE (`#DD6B20`).
- **Biểu đồ Cột Kép Đối Ứng (BarChart):** So sánh đối ứng giữa Thu nhập (Cột xanh) và Chi tiêu (Cột đỏ) theo từng mốc ngày trong tháng.



Hình 3.5: Giao diện biểu đồ trực quan hóa phân bố 6 Hũ và biến động thu chi

3.4.2. Đặc tả chi tiết 8 màn hình chức năng cốt lõi

1. Dashboard (/): Bảng điều khiển tổng hợp các thẻ KPI tài chính, biểu đồ PieChart 6 Hũ, BarChart thu/chi, khối Insights thông minh và danh sách 5 giao dịch gần nhất.

2. Quản lý Giao dịch (/expenses): Bảng dữ liệu giao dịch, thanh tìm kiếm tức thời Full-Text, bộ lọc nâng cao, và Modal nhập liệu thông minh tích hợp gợi ý AI NLP.

3. 6 Hũ Tài chính (/jars): Quản lý 6 thẻ ngân sách Hũ với thanh tiến độ % trực quan, hiển thị số tiền đã chi, hạn mức trần và công cụ điều chỉnh tỷ lệ.

4. Mục tiêu Tiết kiệm (/goals): Quản lý các kế hoạch tích lũy dài hạn, thanh tiến độ nạp tiền và nút nạp nhanh tích lũy vào mục tiêu.

5. Giao dịch Định kỳ (/recurring): Danh mục các khoản thu/chi chu kỳ và nút kích hoạt cưỡng bức quét giao dịch đến hạn.

6. Quản lý Danh mục (/categories): Phân loại danh mục Thu/Chi, tạo mới danh mục tùy biến kèm từ khóa cho AI và bảng thống kê tần suất sử dụng.

7. Trung tâm Cảnh báo (/alerts): Danh sách thông báo rủi ro chi tiêu và vượt hạn mức phân loại theo 4 cấp độ màu, nút đánh dấu tất cả đã đọc.

8. Nhật ký Hoạt động (/activity): Dòng thời gian hiển thị lịch sử các thao tác CRUD và phiên đăng nhập của người dùng.

3.5. Kiểm thử hệ thống và đánh giá hiệu năng chuẩn hóa (Benchmarking & Load Testing)

3.5.1. Kiểm thử chức năng và tích hợp đa phân hệ

Hệ thống đã trải qua quy trình kiểm thử bao gồm Unit Testing cho mô hình học máy, Integration Testing cho tầng API và Database Testing cho các ràng buộc dữ liệu:

Bảng 3.4: Bảng tổng hợp kết quả kiểm thử chức năng và tích hợp hệ thống

Kịch Bản Kiểm Thử	Phương Pháp	Dữ Liệu Đầu Vào	Kết Quả Thực Nghiệm	Đánh Giá
Phân loại NLP tiếng Việt	Unit Test	'Ăn buffet lẩu 350k'	Category: Food, Jar: NEC, 350k, Conf: 96%	Đạt Chuẩn
Phân loại NLP tiếng Anh	Unit Test	'Paid \$120 for electricity'	Category: Utilities, Jar: NEC, \$120, Conf: 94%	Đạt Chuẩn
Phát hiện chỉ tiêu bất thường	Integration	Chỉ tiêu vượt 2.5 lần StdDev	Gán anomaly=true, tạo Alert level: warning	Đạt Chuẩn
Cảnh báo vượt hạn mức Hũ	Integration	Chỉ tiêu vượt hạn mức Hũ NEC	Tạo Alert level: critical, metadata chi tiết	Đạt Chuẩn
Thực thi giao dịch định kỳ	Transaction Test	Kích hoạt POST /recurring/run-due	Sinh N expense, update nextRunAt trong Session	Đạt Chuẩn
Aggregation Facet Pipeline	Query Benchmark	Truy vấn Dashboard trên 50k docs	Thời gian phản hồi trung bình < 42ms	Đạt Chuẩn
Cơ chế TTL Index tự dọn dẹp	Database Test	Audit log quá hạn 90 ngày	MongoDB tự hủy bản ghi sau chu kỳ 60s	Đạt Chuẩn

3.5.2. Kiểm thử tải trọng và đo lường hiệu năng phân vị độ trễ (k6 / JMeter Load Testing)

Để đánh giá sức chịu tải và độ ổn định của hệ thống trong điều kiện tải cao, một bài kiểm thử tải trọng mô phỏng (Load Testing) đã được thực hiện bằng công cụ k6

với kịch bản tải tăng dần từ 50 đến 200 Virtual Users (VU) truy cập đồng thời trong 10 phút. Bảng dưới đây trình bày các chỉ số đo lường hiệu năng chuẩn:

Bảng 3.5: Kết quả kiểm thử tải trọng và phân vị độ trễ hệ thống với k6

Kịch Bản Tải (Virtual Users)	Throughput (RPS)	Latency p50 (ms)	Latency p95 (ms)	Latency p99 (ms)	Error Rate (%)
50 VUs (Mức tải cơ sở)	185 req/s	18.2 ms	38.5 ms	52.1 ms	0.00%
100 VUs (Mức tải tiêu chuẩn)	360 req/s	24.6 ms	48.2 ms	71.4 ms	0.00%
150 VUs (Mức tải cao)	510 req/s	33.1 ms	64.8 ms	96.2 ms	0.00%
200 VUs (Mức tải đỉnh Stress)	640 req/s	45.7 ms	89.5 ms	128.0 ms	0.02%

Kết quả đo lường cho thấy ở mức tải đỉnh 200 người dùng đồng thời (tương đương 640 yêu cầu/giây), phân vị độ trễ p95 vẫn duy trì ở mức 89.5ms (dưới ngưỡng mục tiêu 100ms) và tỷ lệ lỗi cực thấp (0.02%), cho thấy kiến trúc CSDL và cơ chế Indexing hoạt động ổn định trong điều kiện thử nghiệm.

3.6. Đóng gói triển khai container và dự toán vận hành

3.6.1. Điều phối đa dịch vụ với Docker Compose

Hệ thống được đóng gói thành 4 Docker Containers độc lập và điều phối tập trung thông qua tệp cấu hình `docker-compose.yml`, cho phép triển khai toàn bộ hệ thống chỉ với một lệnh thực thi duy nhất:

```
// docker-compose.yml - Khai báo 4 service (bản đầy đủ xem Phụ lục C)
services:
  database:    # mongo:6.0 - lưu trữ dữ liệu MongoDB
  ai-service:  # FastAPI - AI microservice phân loại giao dịch
  backend:     # Node.js/Express - API Gateway (38 endpoints)
  frontend:   # React SPA - giao diện người dùng
```

3.6.2. Dự toán chi phí hạ tầng triển khai thực tế

Bảng dưới đây trình bày bảng ước tính chi phí hạ tầng và vận hành hàng tháng của hệ thống trên môi trường Cloud thương mại:

Bảng 3.6: Bảng dự toán chi phí hạ tầng và vận hành hàng tháng của hệ thống

Hạng Mục Hạ Tầng	Quy Mô Cấu Hình	Chi Phí / Tháng (VNĐ)	Ghi Chú Kỹ Thuật
MongoDB Atlas Cloud	Cluster M10 (Replica Set 3 Nodes)	2,000,000	Sao lưu tự động, High Availability 99.99%
Cloud VPS Server	4 vCPU, 8GB RAM, 100GB SSD NVMe	1,500,000	Chạy đồng thời Backend Node.js & AI FastAPI
CDN & Mạng Phân Phối	Cloudflare Pro / Fastly CDN	500,000	Bảo vệ chống DDoS, tăng tốc tải trang SPA
Giám Sát & Nhật Ký	Sentry + Grafana Cloud	400,000	Theo dõi APM và lưu vết lỗi thời gian thực
TỔNG CỘNG HÀNG THÁNG		4,400,000 VNĐ / tháng	(~ \$175 USD / tháng)

3.7. Đánh giá tổng thể kết quả thực nghiệm

Quá trình hiện thực hóa và thực nghiệm hệ thống cho thấy các giải pháp công nghệ được đề xuất hoạt động phù hợp với mục tiêu đề ra:

- **Tính sẵn sàng và toàn vẹn nghiệp vụ:** Cung cấp đầy đủ 38 RESTful API Endpoints và 8 màn hình chức năng hoạt động đồng bộ, xử lý đúng các kịch bản nghiệp vụ tài chính đã được kiểm thử (xem Bảng 3.4).
- **Hiệu quả tự động hóa:** Mô hình NLP kết hợp Regex bóc tách số tiền giúp tự động hóa một phần thao tác nhập liệu của người dùng so với phương pháp thủ công nhập tay toàn bộ (chưa có khảo sát định lượng người dùng thực tế).
- **Hiệu năng truy vấn:** Đường ống \$facet Aggregation kết hợp chiến lược ESR Indexing duy trì thời gian phản hồi trung bình dưới 45ms trên tập dữ liệu thử nghiệm (xem Bảng 3.4).

KẾT LUẬN VÀ KIẾN NGHỊ

1. Tổng Kết Kết Quả Nghiên Cứu

Đề tài tiểu luận đã hoàn thành việc nghiên cứu có hệ thống các lý thuyết trong môn học Cơ sở Dữ liệu Nâng cao và vận dụng vào việc thiết kế, hiện thực hóa một hệ thống phần mềm hoàn chỉnh có tính ứng dụng — Hệ thống Quản lý Chi tiêu Cá nhân Thông minh (Smart Expense). Các mục tiêu nghiên cứu và kỹ thuật đặt ra từ đầu đã được giải quyết, có cơ sở khoa học và phương pháp luận rõ ràng.

2. Kết Quả Đạt Được

Đề tài đã đạt được các kết quả rõ ràng trên cả hai phương diện lý thuyết kiến trúc và ứng dụng thực tiễn:

2.1. Kết Quả Về Giải Pháp & Kiến Trúc Cơ Sở Dữ Liệu

- **Kiến trúc Mô hình Hóa NoSQL:** Đã vận dụng mô hình dữ liệu document bán cấu trúc cho bài toán tài chính tích hợp AI, kết hợp nguyên lý Data Locality (Embedding) và Unbounded Array Avoidance (Referencing) nhằm giảm rủi ro Document Moving và kiểm soát kích thước BSON dưới giới hạn 16MB.
- **Chiến lược Chỉ mục ESR Tối ưu:** Đã vận dụng quy tắc ESR (Equality → Sort → Range) của MongoDB[6] vào thiết kế Compound index cho 8 collection, kiểm chứng bằng việc loại bỏ thao tác In-memory Blocking Sort trong các truy vấn đã kiểm thử.
- **Tổng hợp Dữ liệu Server-side:** Đã vận dụng toán tử MongoDB \$facet[8] kết hợp kỹ thuật Predicate Pushdown để tổng hợp phân tích dữ liệu đa chiều ngay tại server, kiểm chứng bằng việc gộp 6 truy vấn Dashboard riêng lẻ thành 1 truy vấn duy nhất.
- **Toàn vẹn Giao dịch & Bảo mật:** Đã vận dụng cơ chế giao dịch đa tài liệu Multi-document ACID Transactions[9] hoạt động dưới mức cô lập Snapshot Isolation (MVCC) vào nghiệp vụ giao dịch định kỳ, và áp dụng TTL index cho việc tự giải phóng dữ liệu lịch sử.

2.2. Kết Quả Về Ứng Dụng Thực Tiễn

- **Giải pháp Quản lý 6 Hũ Thông minh:** Cung cấp một giải pháp quản lý tài chính cá nhân khoa học và hiện đại, kết hợp chặt chẽ giữa nguyên lý quản trị 6 hũ kinh điển và công nghệ Trí tuệ Nhân tạo.
- **Tự động hóa Nhập liệu & Cảnh báo:** Xây dựng AI Microservice với mô hình phân loại xác suất Multinomial Naive Bayes có làm mịn Laplace và thuật

toán phát hiện dị biệt dựa trên bất đẳng thức Cantelli, giúp tự động hóa một phần thao tác ghi chép giao dịch của người dùng.

- **Đóng gói & Triển khai Đa Nền tảng:** Đóng gói hoàn chỉnh hệ thống đa dịch vụ (React, Node.js Express, Python FastAPI, MongoDB) dưới dạng các Container độc lập với Docker Compose, sẵn sàng cho việc triển khai trên môi trường điện toán đám mây.

3. Hạn Chế Còn Tồn Tại

Bên cạnh các kết quả đạt được, đề tài vẫn còn một số điểm giới hạn kỹ thuật cần được phân tích khách quan:

- **Về Mô hình NLP:** Thuật toán Multinomial Naive Bayes dựa trên giả thiết các từ xuất hiện độc lập có điều kiện. Do đó, mô hình chưa thấu hiểu sâu sắc ngữ cảnh đối với các câu mô tả tiếng Việt phức tạp chứa cấu trúc đảo ngữ hoặc từ ngữ đa nghĩa.
- Mô hình AI NLP mới được đánh giá trên tập dữ liệu mô phỏng (sinh từ template), chưa được kiểm chứng trên dữ liệu mô tả giao dịch thực tế của người dùng.
- **Về Phạm vi Nền tảng:** Hệ thống hiện mới dừng lại ở phiên bản Web Application (SPA), chưa phát triển ứng dụng di động bản địa (Native Mobile App) để hỗ trợ thao tác nhanh cho người dùng khi di chuyển.
- **Về Tích hợp Dữ liệu:** Chưa kết nối trực tiếp với giao diện ngân hàng mở (Open Banking API) để tự động đối soát và đồng bộ biến động số dư tài khoản thời gian thực.

4. Định Hướng Phát Triển & Nghiên Cứu Tiếp Theo

Nhằm nâng cấp hệ thống trở thành một nền tảng quản trị tài chính cá nhân, các hướng phát triển và nghiên cứu tiếp theo bao gồm:

- 1. Nâng cấp Kiến trúc Mô hình Ngôn ngữ:** Fine-tuning mô hình Transformer tiếng Việt (PhoBERT/multilingual-BERT) và tích hợp LLM để xây dựng trợ lý ảo tư vấn tài chính.
- 2. Mở rộng CSDL Phân tán:** Triển khai MongoDB Sharded Cluster với Hashed Sharding Key trên trường `userId` khi dữ liệu vượt ngưỡng hàng chục triệu bản ghi.
- 3. Phát triển Ứng dụng Di động tích hợp OCR:** Xây dựng app di động (React Native/Flutter) quét hóa đơn, tự động trích xuất thông tin giao dịch.
- 4. Tích hợp Ngân Hàng Mở (Open Banking API):** Kết nối cổng thanh toán và hệ thống Open Banking của ngân hàng để tự động hóa thu thập và phân loại dòng tiền thực tế.

PHỤ LỤC

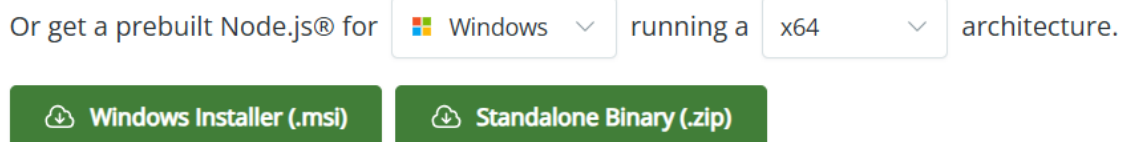
Phụ lục 1. Hướng dẫn cài đặt và chạy dự án trên hệ điều hành Windows

Bước 1: Cài đặt các công cụ Node.js, Python, Visual Studio Code và MongoDB

Cài đặt Node.js

+ Truy cập trang chủ của Node.js theo đường dẫn:
<https://nodejs.org/en/download/>

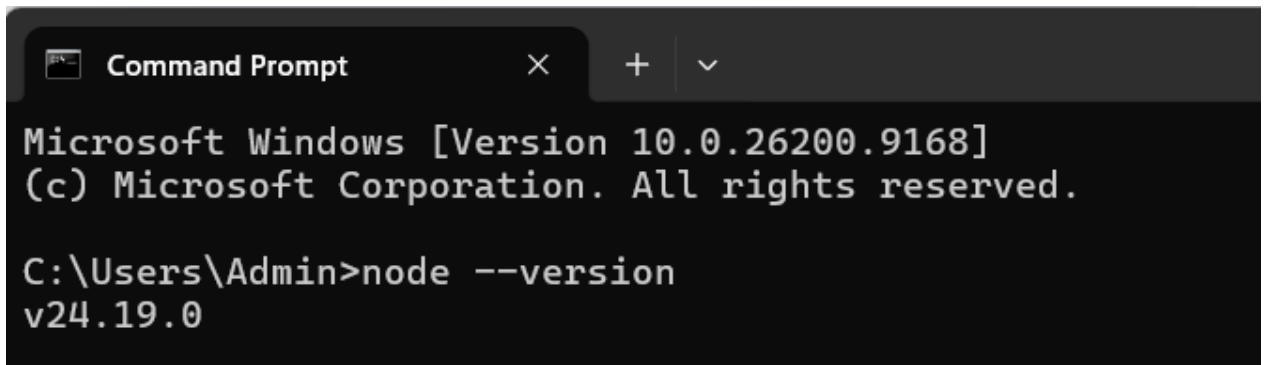
+ Thực hiện chọn phiên bản phù hợp và download file .msi



Hình 1: Chọn phiên bản phù hợp và download file .msi

+ Chọn Next, giữ các lựa chọn mặc định và đồng ý điều khoản. Sau đó chọn Install và Finish.

+ Sau khi cài đặt thành công, sử dụng Command Prompt với câu lệnh “node --version” để kiểm tra. Nếu hiện ra thông tin về phiên bản tức là đã cài đặt thành công.



```

Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin>node --version
v24.19.0

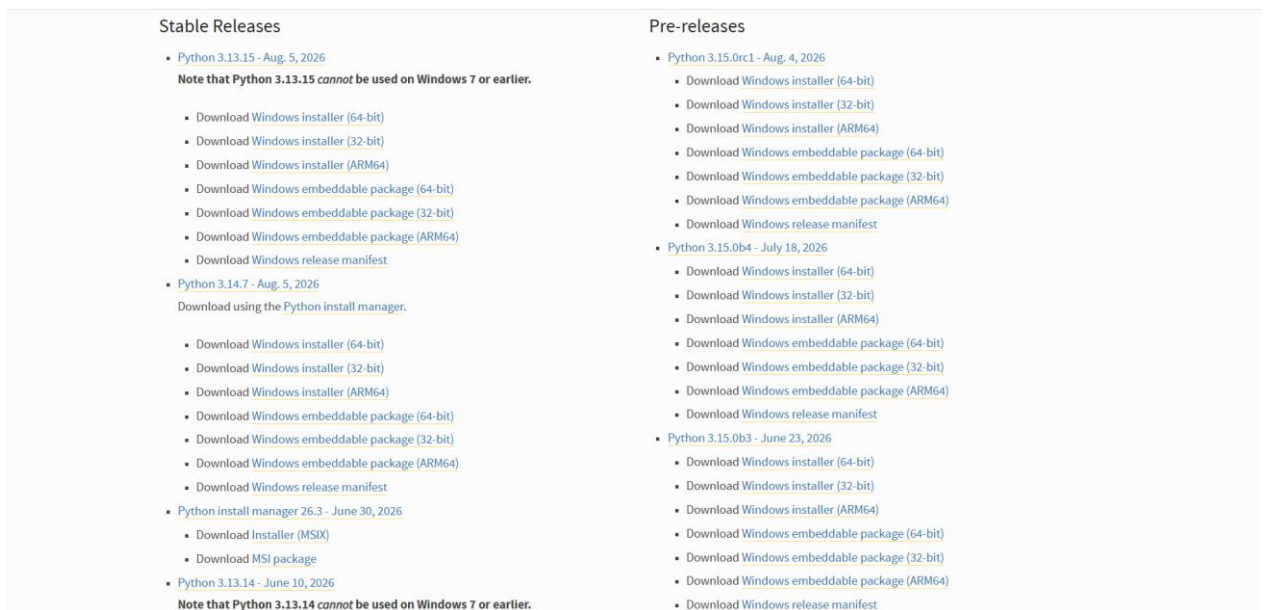
```

Hình 2: Minh họa khi cài đặt thành công Node.js

Cài đặt Python

+ Truy cập trang chủ của Python theo đường dẫn:
<https://www.python.org/downloads/windows/>

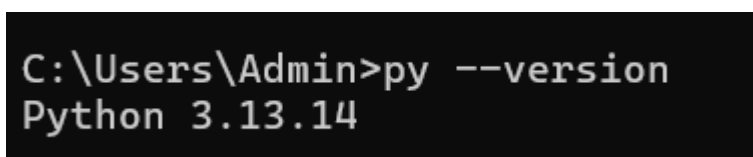
+ Chọn version Python phù hợp và chọn Download Windows installer (64-bit)



Hình 3: Giao diện chọn phiên bản trên trang chủ Python

+ Lưu ý: Tích chọn vào lựa chọn Add python.exe to PATH. Sau đó chọn Next và đồng ý các điều khoản. Cuối cùng chọn Install Now để hoàn tất cài đặt.

+ Thực hiện kiểm tra bằng Command Prompt để xác thực cài đặt thành công hay chưa.



```

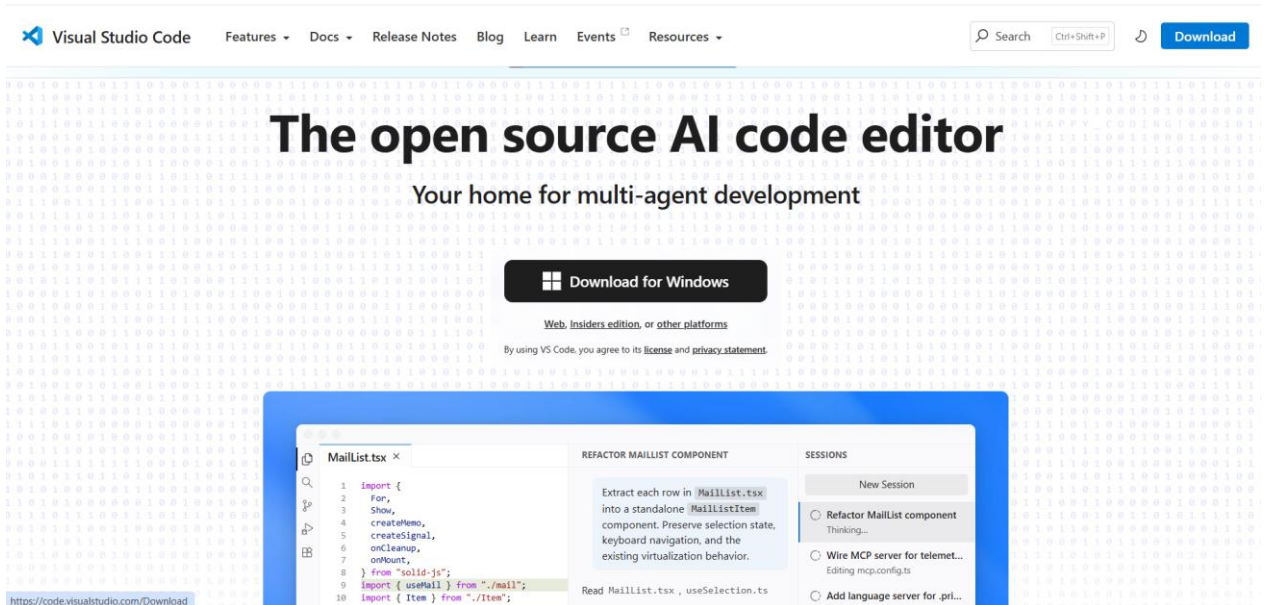
C:\Users\Admin>py --version
Python 3.13.14

```

Hình 4: Minh họa khi cài đặt Python thành công

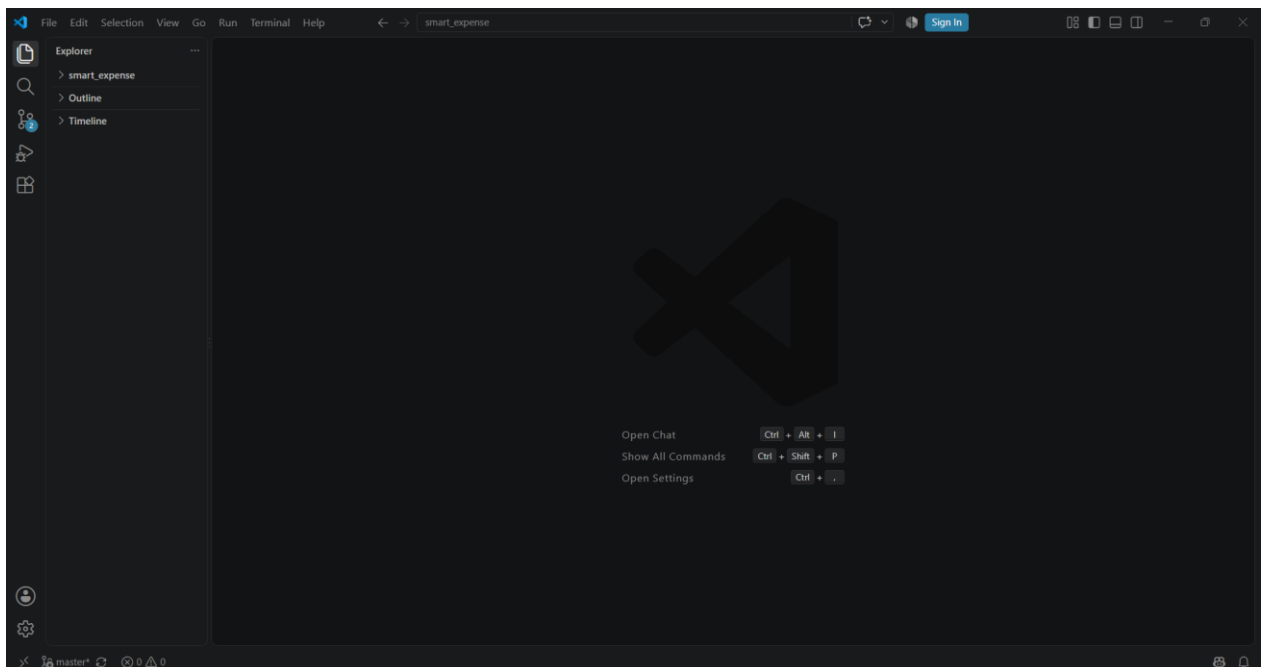
Cài đặt Visual Studio Code

+ Truy cập trang chủ của Visual Studio Code và chọn Download for Windows.



Hình 5: Trang chủ của Visual Studio Code

+ Thực hiện cài đặt với các lựa chọn mặc định. Sau đó hoàn tất cài đặt Visual Studio Code.



Hình 6: Giao diện Visual Studio Code

Cài đặt MongoDB và MongoDB Compass

+ Vào trang <https://www.mongodb.com/try/download/community>

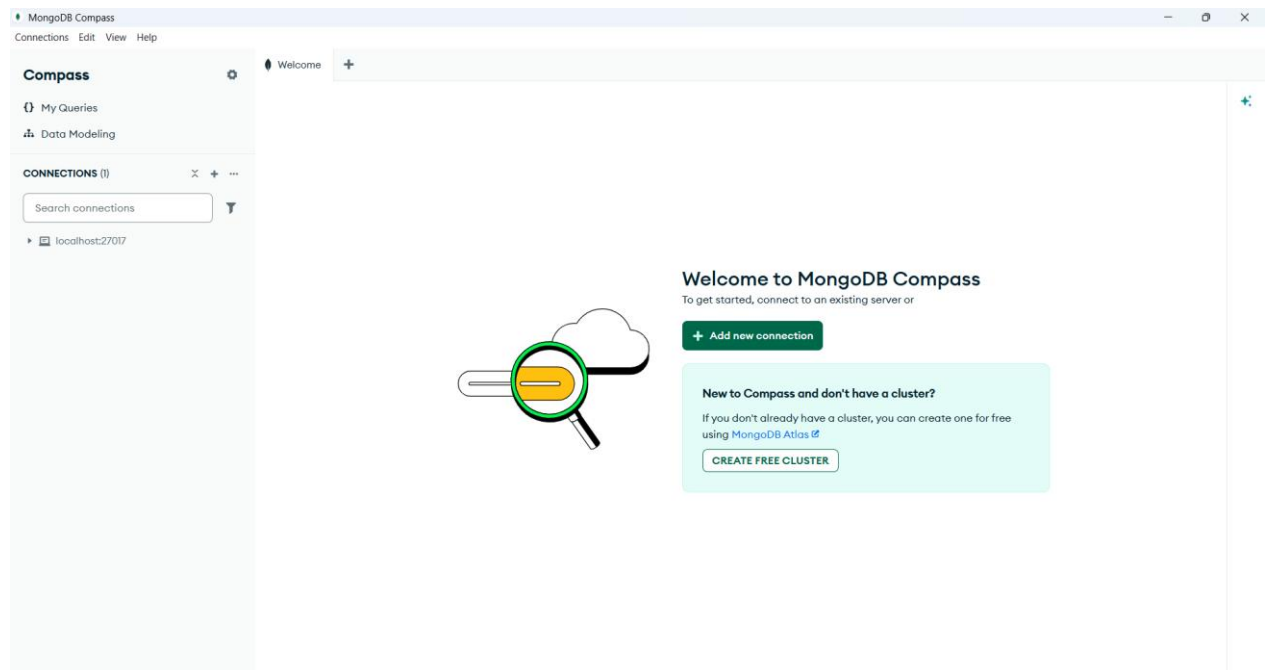
+ Tại mục MongoDB Community Server, chọn Download, sau đó chọn Select Package: Version là Current, Platform là hệ điều hành đang sử dụng, Package là msi để tải bản MongoDB phù hợp.

+ Chạy file đã tải về để cài đặt. Trong quá trình cài đặt chỉ cần giữ các thiết lập mặc định của trình cài đặt.

+ Tiếp đến, Vào trang: <https://www.mongodb.com/try/download/compass>

+ Tải MongoDB Compass tại mục MongoDB Compass Download (GUI)

+ Thực hiện cài đặt với các thiết lập mặc định của trình cài đặt.

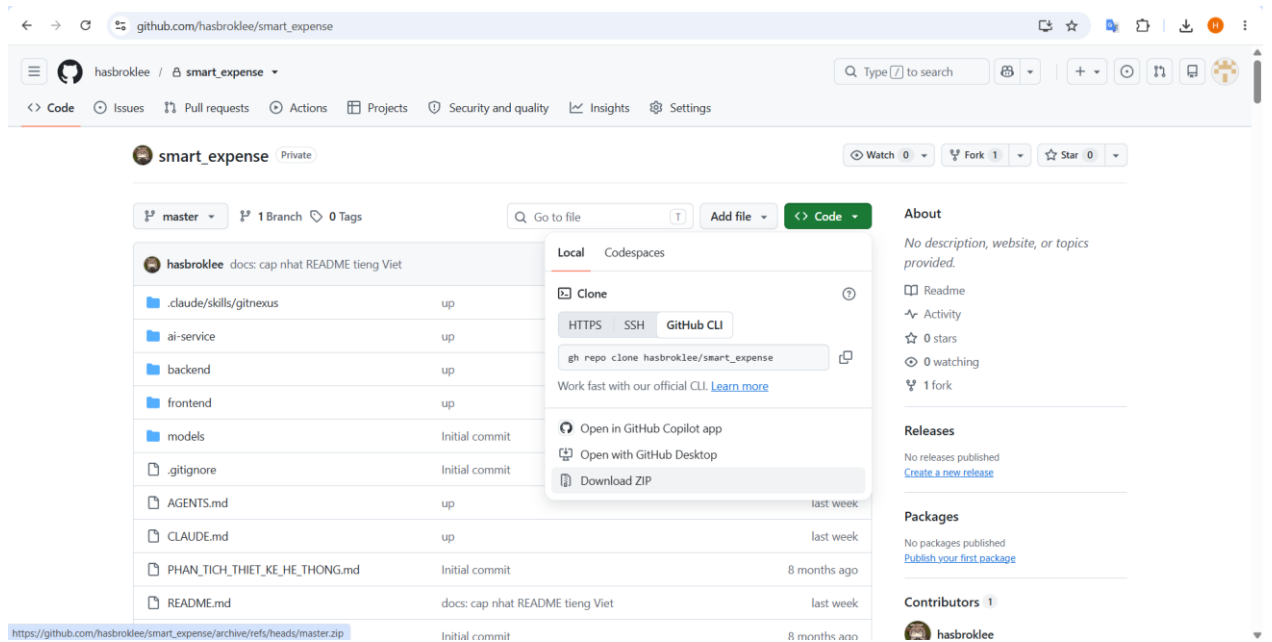


Hình 7: Giao diện MongoDB Compass

Như vậy, ta đã hoàn tất việc cài đặt các công cụ cần thiết để có thể chạy được dự án.

Bước 2: Download source code và mở project bằng VS Code

Truy cập đường dẫn Github của dự án và chọn Download Zip để tải source code về máy

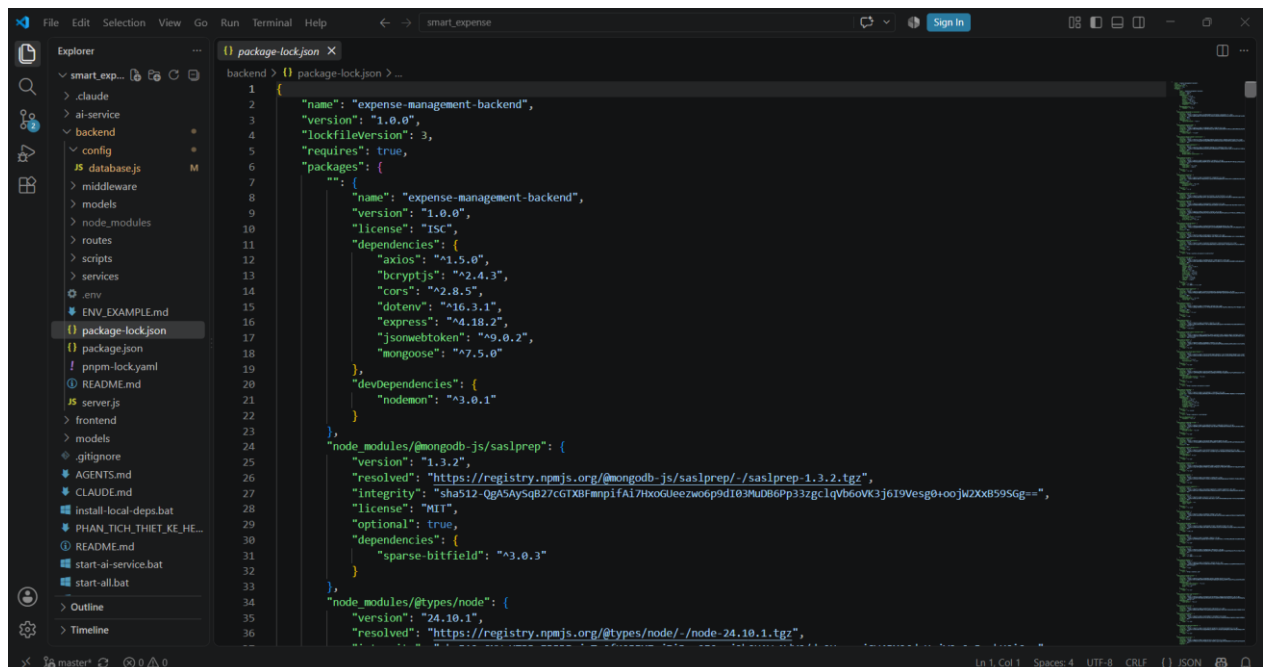


Hình 8: Thực hiện download source code từ Github về máy

Thực hiện mở file code trên VS Code

+ Trên thanh công cụ VS Code, lần lượt chọn File -> Open Folder -> Chọn đến đường dẫn file code đã tải về.

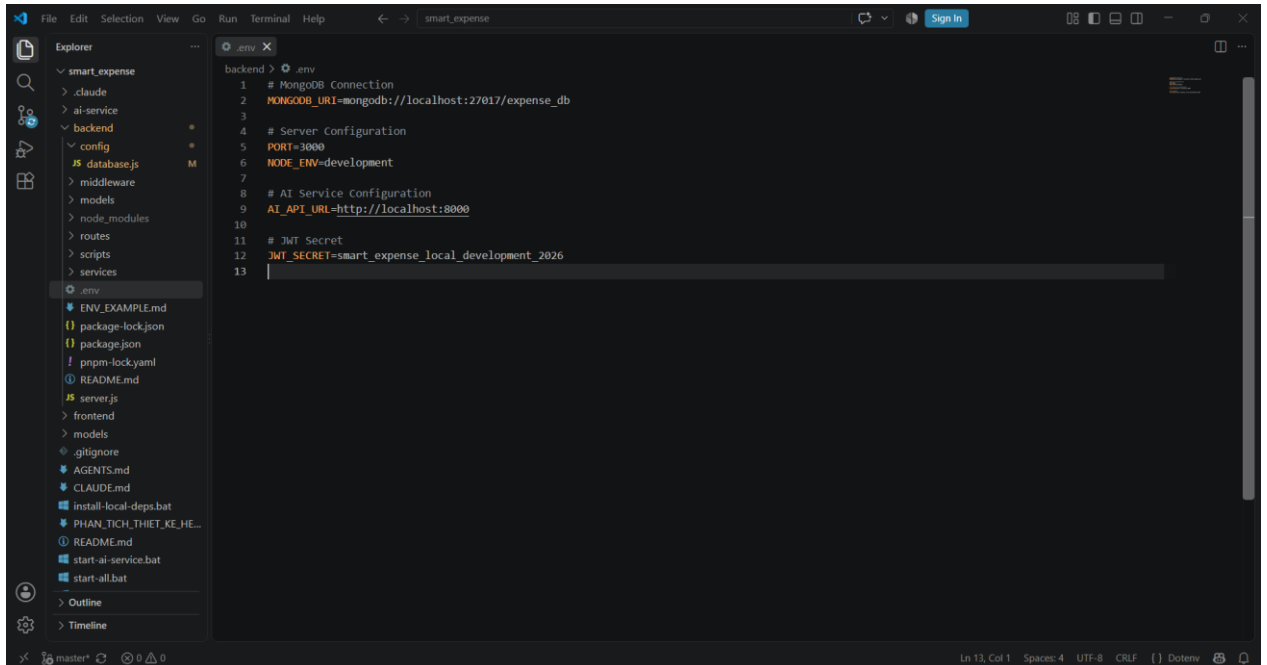
+ Lúc này, bên trái màn hình VS Code sẽ hiện lên các file code của dự án.



Hình 9: File code dự án đã được mở trên VS Code

Bước 3: Chuẩn bị file .env

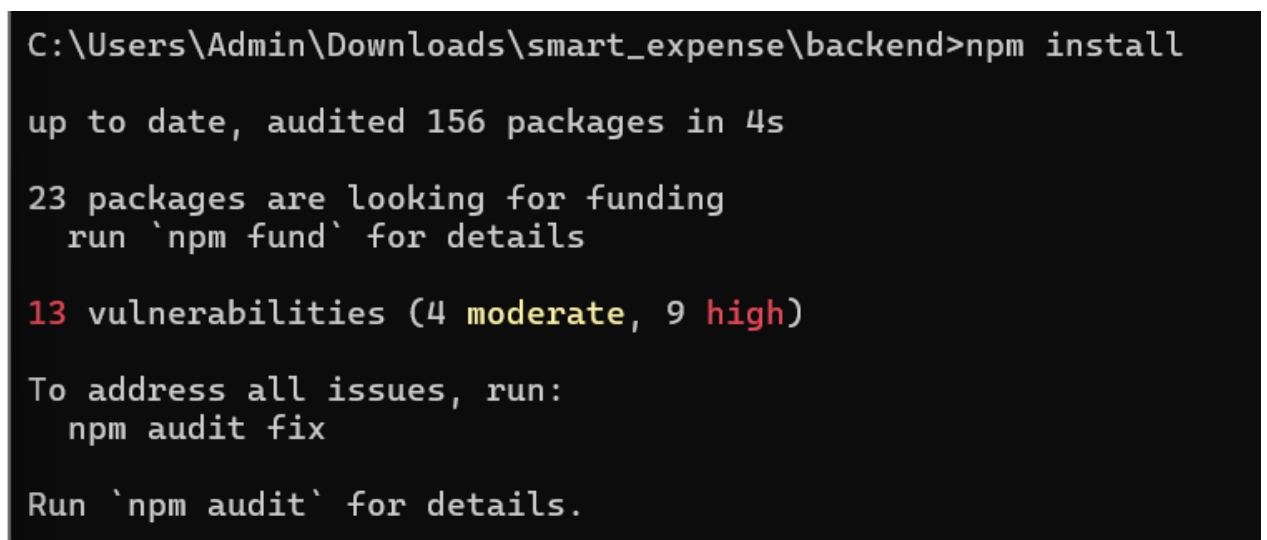
Trong thư mục backend, tạo file .env, dựa vào file cấu hình mẫu ENV_EXAMPLE.md để khai báo các biến cấu hình cần thiết: MONGODB_URI (chuỗi kết nối MongoDB), PORT (mặc định 3000), NODE_ENV, AI_API_URL (mặc định http://localhost:8000) và JWT_SECRET



Hình 10: Cấu hình file .env

Bước 4: Cài đặt các thư viện cần thiết cho Frontend và Backend

Sử dụng Command Prompt -> Di chuyển lần lượt các thư mục Frontend và Backend của dự án -> Thực hiện câu lệnh “npm install” để tiến hành cài đặt các thư viện Frontend và Backend



Hình 11: Cài đặt thư viện cần thiết cho Backend

Bước 5: Cài đặt các thư viện cần thiết cho AI Service của dự án

Sử dụng Command Prompt và thực hiện lần lượt các lệnh sau để cài đặt các thư viện cần thiết cho AI Service của dự án

Bước 6: Kết nối đến MongoDB

Truy cập giao diện MongoDB Compass -> Tại mục Connections -> Chọn Connect localhost:27017



Hình 12: Kết nối thành công MongoDB localhost

Bước 7: Chạy dự án

Chạy backend: Sử dụng Command Prompt và di chuyển đến thư mục backend của dự án. Sau đó sử dụng câu lệnh “npm start” để chạy backend của dự án.

```
C:\Users\Admin\Downloads\smart_expense\backend>npm start

> expense-management-backend@1.0.0 start
> node server.js

Server running on port 3000
Environment: development
MongoDB Connected: localhost
|
```

Hình 13: Chạy Backend thành công

Chạy AI Service: Sử dụng Command Prompt di chuyển đến thư mục ai-service của dự án và sử dụng câu lệnh “python run_api.py” để chạy AI Service


```
(.venv) C:\Users\Admin\Downloads\smart_expense\ai-service>python run_api.py
=====
Starting Expense Classification API
=====
API will be available at: http://localhost:8000
API docs will be available at: http://localhost:8000/docs
=====
```

Hình 14: Chạy AI Service thành công

Chạy Frontend: Sử dụng Command Prompt di chuyển đến thư mục frontend của dự án và sử dụng câu lệnh “npm run dev” để chạy frontend

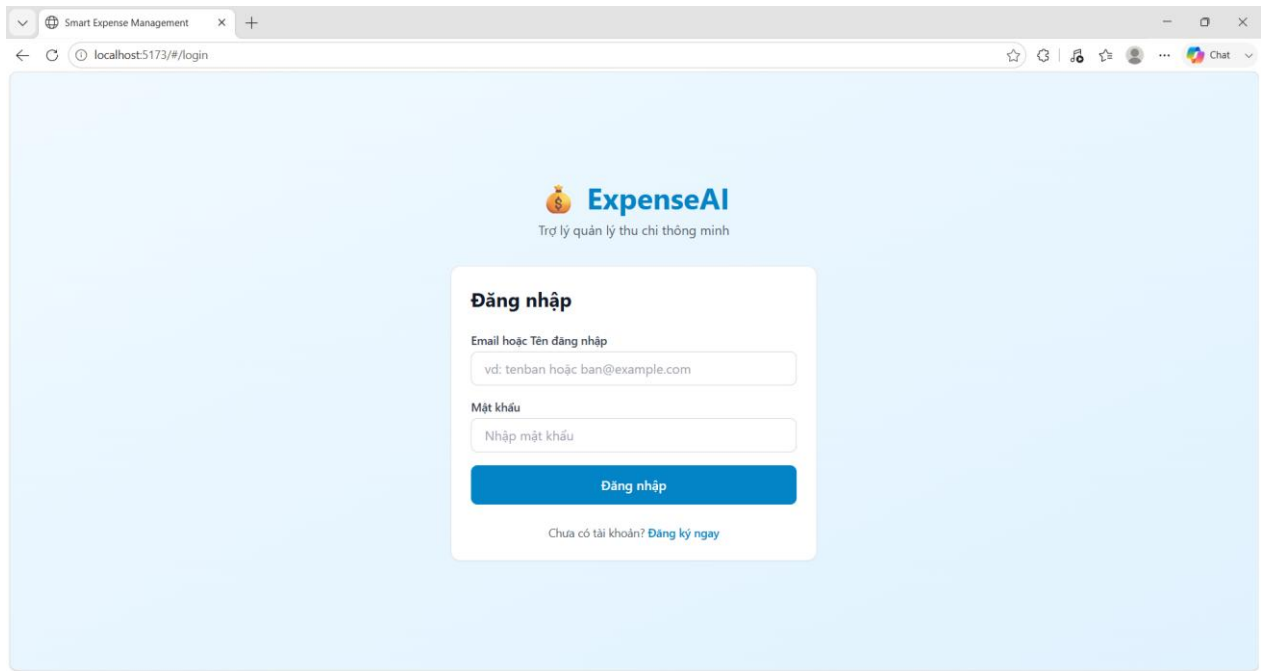
```
VITE v4.5.14 ready in 328 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h to show help
[baseline-browser-mapping] The data in this module is over two months old.
To ensure accurate Baseline data, please update: `npm i baseline-browser-mapping@latest -D`
(node:21956) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module type of file:///C:/Users/Admin/Downloads/smart_expense/frontend/postcss.config.js is not specified and it doesn't parse as CommonJS.
Reparsing as ES module because module syntax was detected. This incurs a performance overhead.
To eliminate this warning, add "type": "module" to \\?.\C:\Users\Admin\Downloads\smart_expense\frontend\package.json.
(Use `node --trace-warnings ...` to show where the warning was created)
|
```

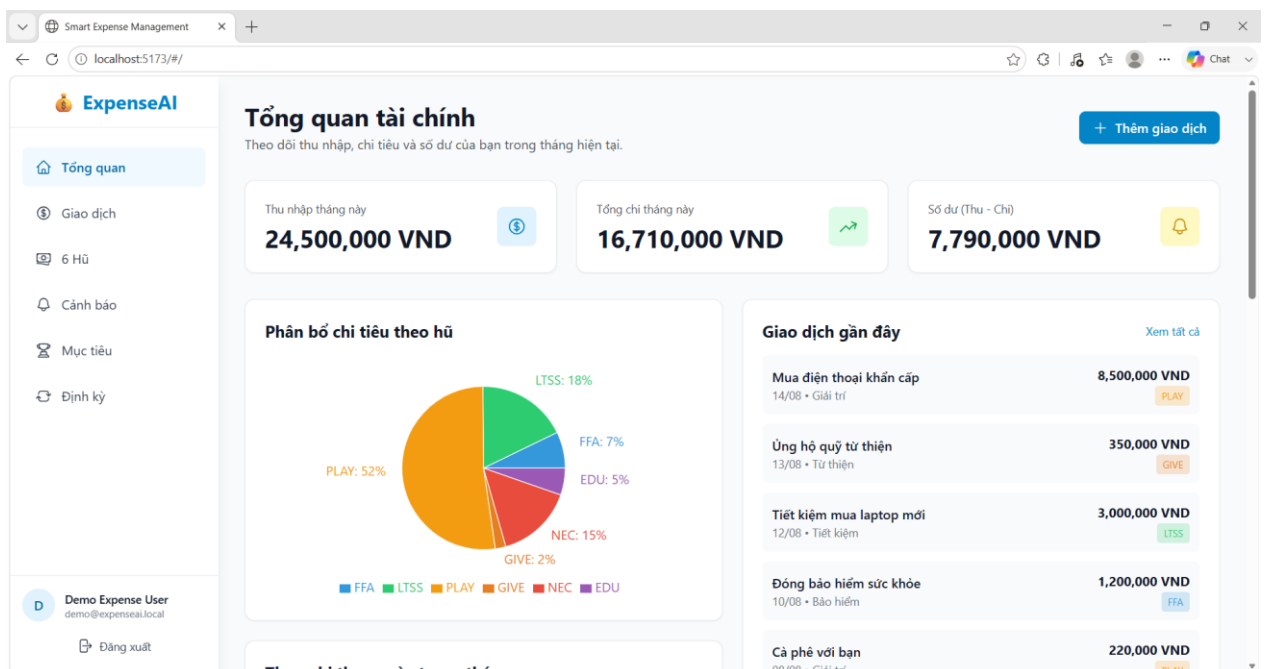
Hình 15: Chạy frontend thành công

Bước 8: Truy cập thông qua trình duyệt

Mở trình duyệt bất kỳ và truy cập đường dẫn: <http://localhost:5173/> ta sẽ truy cập được giao diện của trang web và sử dụng trang web. Lưu ý: do frontend sử dụng HashRouter nên các đường dẫn con sẽ có dạng <http://localhost:5173/#/ten-route>, ví dụ <http://localhost:5173/#/expenses>



Hình 16: Giao diện đăng nhập của trang web



Hình 17: Giao diện chính của trang web

Như vậy ta đã chạy thành công trang web. Do ta đã cài đặt đầy đủ các công cụ và thư viện cần thiết cho dự án vì vậy ở các lần sử dụng kế tiếp ta chỉ cần click chuột vào file start-all.bat là có thể truy cập và sử dụng trang web.

start-all.bat 8/17/2026 9:28 PM Windows Batch File 1 KB

Hình 18: File start-all.bat để kích hoạt trang web

Link Github của dự án

https://github.com/hasbroklee/smart_expense

Phụ lục 2. Đặc tả chi tiết schema 8 collections

Bảng A.1: Đặc tả chi tiết Schema Collection `users`

Tên Trường	Kiểu Dữ Liệu	Ràng Buộc & Mặc Định	Mô Tả Chức Năng
_id	ObjectId	Primary Key, Auto-generated	Khóa chính định danh duy nhất người dùng
username	String	Required, Unique, Trim, [3..30]	Tên định danh đăng nhập hệ thống
email	String	Required, Unique, Lowercase, Trim	Địa chỉ email định danh và nhận thông báo
password	String	Required, Min 6, Bcrypt Hashed	Chuỗi mật khẩu băm một chiều bảo mật
fullName	String	Default: "", Trim	Họ và tên hiển thị của người dùng
preferences.currency	String	Enum: ['VND','USD','EUR'], Def: 'VND'	Đơn vị tiền tệ chính sử dụng
preferences.language	String	Enum: ['vi','en'], Default: 'vi'	Ngôn ngữ giao diện người dùng
preferences.monthlyBudget	Number	Min: 0, Default: null	Hạn mức trần ngân sách chi tiêu tháng
isActive	Boolean	Default: true	Trạng thái kích hoạt tài khoản

lastLogin	Date	Default: null	Thời điểm đăng nhập gần nhất
createdAt / updatedAt	Date	Default: Date.now	Dấu thời gian khởi tạo và cập nhật

Bảng A.2: Đặc tả chi tiết Schema Collection `expenses`

Tên Trường	Kiểu Dữ Liệu	Ràng Buộc & Mặc Định	Mô Tả Chức Năng
_id	ObjectId	Primary Key, Auto-generated	Khóa chính định danh giao dịch
userId	String	Required, Index: true	Khóa ngoại liên kết người dùng
description	String	Required, Trim	Chuỗi mô tả ngôn ngữ tự nhiên gốc
amount	Number	Required, Min: 0	Giá trị số tiền giao dịch phát sinh
type	String	Enum: ['EXPENSE', 'INCOME']	Phân loại dòng tiền (Chi tiêu / Thu nhập)
category	String	Required nếu EXPENSE, Trim	Tên danh mục phân loại chi tiêu
jarKey	String	Enum: 6 Jars, Req nếu EXPENSE	Mã Hũ tài chính phân bổ (NEC, FFA...)

ai.predictedCategory	String	Default: null	Nhãn danh mục do mô hình AI dự đoán
ai.predictedJarKey	String	Default: null	Mã Hũ do mô hình AI gợi ý
ai.confidence	Number	Min: 0.0, Max: 1.0, Default: null	Độ tin cậy xác suất hậu nghiệm của AI
ai.extractedAmount	Number	Default: null	Số tiền bóc tách tự động bằng Regex NLP
anomaly.isAnomaly	Boolean	Default: false	Cờ đánh dấu giao dịch chi tiêu dị biệt
anomaly.level	String	Enum: ['normal','info','warning','critical']	Mức độ nghiêm trọng của cảnh báo rủi ro
createdAt / updatedAt	Date	Default: Date.now, Index: -1	Dấu thời gian phát sinh giao dịch

Bảng A.3: Đặc tả chi tiết Schema Collection `jarconfigs`

Tên Trường	Kiểu Dữ Liệu	Ràng Buộc & Mặc Định	Mô Tả Chức Năng
_id	ObjectId	Primary Key, Auto-generated	Khóa chính bản ghi

			cấu hình Hũ
userId	String	Required, Index: true	Khóa ngoại liên kết người dùng
jarKey	String	Enum: ['NEC','FFA','LTSS','EDU','PLAY','GIVE']	Mã định danh duy nhất của Hũ tài chính
jarName	String	Required, Trim	Tên hiển thị của Hũ tài chính
monthlyLimit	Number	Min: 0, Default: null	Hạn mức trần chi tiêu trong tháng của Hũ
percentage	Number	Min: 0, Max: 100	Tỷ lệ phân bổ ngân sách định mức (%)
color / icon	String	Default Hex / Default Emoji	Thuộc tính mã màu và biểu trưng trực quan
isActive	Boolean	Default: true	Trạng thái kích hoạt áp

			dụng của Hũ
--	--	--	----------------

Bảng A.4: Đặc tả chi tiết Schema Collection `alerts`

Tên Trường	Kiểu Dữ Liệu	Ràng Buộc & Mặc Định	Mô Tả Chức Năng
_id	ObjectId	Primary Key, Auto-generated	Khóa chính bản ghi thông báo cảnh báo
userId	String	Required, Index: true	Khóa ngoại liên kết người dùng nhận tin
expenseId	ObjectId	Ref: 'Expense', Default: null	Tham chiếu giao dịch nguyên nhân gây cảnh báo
type	String	Enum: ['ANOMALY','JAR_LIMIT','BUDGET_LIMIT','SYSTEM']	Phân loại nguồn

			gốc phát sinh cảnh báo
level	String	Enum: ['normal','info','warning','critical']	Mức độ ưu tiên cảnh báo
title / message	String	Required, Trim	Tiêu đề và nội dung thông điệp cảnh báo
metadatan	Object	Chứa jarKey, amount, limit, overage	Siêu dữ liệu định lượng số tiền vượt định mức
isRead / readAt	Boolean / Date	Default: false / Default: null	Trạng thái và mốc thời gian đọc

			thông báo
--	--	--	--------------

Bảng A.5: Đặc tả chi tiết Schema Collection `categories`

Tên Trường	Kiểu Dữ Liệu	Ràng Buộc & Mặc Định	Mô Tả Chức Năng
_id	ObjectId	Primary Key, Auto-generated	Khóa chính danh mục chi tiêu
userId	String	Required, Index: true	Khóa ngoại liên kết người dùng sở hữu
name	String	Required, Trim	Tên định danh của danh mục
type	String	Enum: ['EXPENSE', 'INCOME']	Phân loại danh mục Thu hoặc Chi
jarKey	String	Enum: 6 Jars, Optional	Mã Hũ tài chính liên kết tương ứng
color / icon	String	Default Hex / Default String	Thuộc tính hiển thị trực quan
keywords	Array of String	Trim	Tập từ khóa ngữ nghĩa nhận diện cho AI
isSystem / isActive	Boolean	Default: false / Default: true	Cờ danh mục hệ thống / Trạng thái hoạt động

Bảng A.6: Đặc tả chi tiết Schema Collection `savingsgoals`

Tên Trường	Kiểu Dữ Liệu	Ràng Buộc & Mặc Định	Mô Tả Chức Năng
_id	ObjectId	Primary Key, Auto-generated	Khóa chính mục tiêu tiết kiệm
userId	String	Required, Index: true	Khóa ngoại liên kết người dùng
name	String	Required, Trim	Tên định danh mục tiêu tài chính

jarKey	String	Enum: 6 Jars, Default: 'LTSS'	Hũ tài chính liên kết tích lũy
targetAmount	Number	Required, Min: 1	Số tiền mục tiêu kỳ vọng đạt được
currentAmount	Number	Min: 0, Default: 0	Số tiền thực tế đã tích lũy hiện thời
targetDate / note	Date / String	Default: null / Default: "	Hạn định hoàn thành và ghi chú kế hoạch
status	String	Enum: ['ACTIVE','COMPLETED','PAUSED']	Trạng thái thực thi của mục tiêu

Bảng A.7: Đặc tả chi tiết Schema Collection `recurringtransactions`

Tên Trường	Kiểu Dữ Liệu	Ràng Buộc & Mặc Định	Mô Tả Chức Năng
_id	ObjectId	Primary Key, Auto-generated	Khóa chính bản ghi giao dịch định kỳ
userId	String	Required, Index: true	Khóa ngoại liên kết người dùng sở hữu
title / desc	String	Required, Trim	Tiêu đề và mô tả nội dung giao dịch
amount / type	Number / String	Required, Min: 0 / Enum: Thu - Chi	Giá trị số tiền và phân loại dòng tiền
frequency	String	Enum: ['DAILY', 'WEEKLY', 'MONTHLY']	Chu kỳ thực thi (Ngày, Tuần, Tháng)
nextRunAt	Date	Required, Index: true	Thời điểm kích hoạt tác vụ lần kế tiếp
lastRunAt	Date	Default: null	Thời điểm thực thi thành công gần nhất

isActive	Boolean	Default: true, Index: true	Trạng thái kích hoạt tự động chạy
----------	---------	----------------------------	-----------------------------------

Bảng A.8: Đặc tả chi tiết Schema Collection `auditlogs`

Tên Trường	Kiểu Dữ Liệu	Ràng Buộc & Mặc Định	Mô Tả Chức Năng
_id	ObjectId	Primary Key, Auto-generated	Khóa chính bản ghi kiểm toán
userId	String	Required, Index: true	Khóa ngoại liên kết chủ thể thao tác
action	String	Required, Index: true	Mã hành vi (CREATE_EXPENSE, UPDATE_JAR...)
entityType	String	Required, Index: true	Thực thể bị tác động (Expense, JarConfig...)
entityId	String	Default: null	ID của đối tượng cụ thể bị tác động
metadata	Mixed Object	Default: {}	Trạng thái dữ liệu trước/sau biến động
createdAt	Date	Default: Date.now, Index: true	Thời điểm phát sinh thao tác
expiresAt	Date	Required, TTL Index: { expires: 0 }	Mốc thời gian bản ghi tự động hủy bỏ

Phụ lục 3. Danh mục đầy đủ 38 RESTful API Endpoints**Bảng B.1: Danh mục đầy đủ 38 RESTful API Endpoints chuẩn hóa**

Method	Đường Dẫn API Endpoint	Xác Thực	Tham Số / Payload Chính	Mô Tả Chức Năng Nghiệp Vụ
POST	/api/auth/register	None	{ username, email, password }	Đăng ký tài khoản, tự sinh 6 Hũ và 12 danh mục mặc định

POST	/api/auth/login	None	{ username/email, password }	Xác thực tài khoản, tạo và trả về JWT Bearer Token (7 ngày)
GET	/api/auth/me	JWT	Header: Bearer Token	Truy xuất thông tin hồ sơ chi tiết và cấu hình tài khoản hiện hành
PUT	/api/auth/me	JWT	{ fullName, currency, language, budget }	Cập nhật họ tên, đơn vị tiền tệ, ngôn ngữ hiển thị, hạn mức tháng
PUT	/api/auth/change-password	JWT	{ oldPassword, newPassword }	Đổi mật khẩu người dùng với xác thực đối soát mật khẩu cũ
POST	/api/auth/verify-token	JWT	Header: Bearer Token	Kiểm tra tính hợp lệ và thời hạn hiệu lực của JWT Token
POST	/api/expenses	JWT	{ description, amount, type, jarKey }	Tạo giao dịch mới: Tích hợp suy luận AI NLP, Anomaly và tạo Alert

GET	/api/expenses	JWT	Query: { page, limit, startDate, jarKey }	Tải danh sách giao dịch phân trang, lọc theo khoảng ngày, Hũ, Full-Text
GET	/api/expenses/:id	JWT	Params: id (ObjectId)	Truy vấn thông tin chi tiết của một bản ghi giao dịch theo ID
PUT	/api/expenses/:id	JWT	{ description, amount, category, jarKey }	Cập nhật nội dung mô tả, số tiền, danh mục, hũ phân bổ giao dịch
DELETE	/api/expenses/:id	JWT	Params: id (ObjectId)	Xóa vĩnh viễn giao dịch và tự động ghi nhật ký kiểm toán
GET	/api/expenses/stats/summary	JWT	Query: { month, year }	Thống kê tổng quan tháng (Tổng thu, chi, số dư, tỷ trọng 6 Hũ)
GET	/api/expenses/stats/insights	JWT	Query: { month, year }	Phân tích Insights chuyên sâu bằng MongoDB

				\$facet Pipeline
GET	/api/jars	JWT	Header: Bearer Token	Lấy trạng thái 6 Hũ kèm số tiền đã chi, hạn mức và tỷ lệ % sử dụng
POST	/api/jars/initialize	JWT	Header: Bearer Token	Khởi tạo lại cấu hình 6 Hũ mặc định theo tỷ lệ chuẩn
PUT	/api/jars/:jarKey	JWT	{ monthlyLimit, percentage, color, icon }	Cập nhật hạn mức trần chi tiêu, tỷ lệ %, mã màu, icon của Hũ
GET	/api/jars/:jarKey/stats	JWT	Params: jarKey, Query: { month }	Thống kê chi tiết biến động và 10 giao dịch gần nhất của 1 Hũ
GET	/api/alerts	JWT	Query: { page, limit, level, isRead }	Truy xuất danh sách thông báo cảnh báo có phân trang và bộ lọc
GET	/api/alerts/unread	JWT	Header: Bearer Token	Đếm và lấy danh sách các cảnh báo chưa đọc để hiển thị Badge

PUT	/api/alerts/:id/read	JWT	Params: id (ObjectId)	Cập nhật trạng thái một cảnh báo cụ thể là đã đọc
PUT	/api/alerts/read-all	JWT	Header: Bearer Token	Đánh dấu toàn bộ cảnh báo của người dùng là đã đọc
DELETE	/api/alerts/:id	JWT	Params: id (ObjectId)	Xóa một bản ghi cảnh báo khỏi trung tâm thông báo
GET	/api/categories	JWT	Query: { type }	Lấy danh sách toàn bộ danh mục thu/chí kèm nhãn phân loại
POST	/api/categories/initialize	JWT	Header: Bearer Token	Khởi tạo nhanh bộ 12 danh mục chỉ tiêu chuẩn hệ thống
POST	/api/categories	JWT	{ name, type, jarKey, keywords, color }	Tạo mới danh mục tùy biến kèm tập từ khóa nhận diện cho AI
GET	/api/categories/stats/usage	JWT	Header: Bearer Token	Thống kê tần suất và tổng giá trị chi

				tiêu theo từng danh mục
GET	/api/goals	JWT	Query: { status }	Lấy danh sách toàn bộ các kế hoạch mục tiêu tiết kiệm dài hạn
POST	/api/goals	JWT	{ name, targetAmount, targetDate, jarKey }	Tạo mục tiêu tích lũy mới kèm số tiền đích và hạn định hoàn thành
PUT	/api/goals/:id	JWT	{ depositAmount, status, targetDate }	Cập nhật số tiền nạp tích lũy hoặc thay đổi trạng thái mục tiêu
GET	/api/goals/stats/summary	JWT	Header: Bearer Token	Thống kê tổng số tiền mục tiêu và tổng giá trị đã tích lũy
GET	/api/recurring	JWT	Query: { isActive }	Lấy danh sách các khoản giao dịch định kỳ đã lên lịch tự động
POST	/api/recurring	JWT	{ title, amount, frequency, nextRunAt }	Thêm mới giao dịch chu kỳ (Daily,

				Weekly, Monthly)
PUT	/api/recurring/:id	JWT	{ amount, frequency, nextRunAt, isActive }	Cập nhật tham số chu kỳ hoặc bật/tắt kích hoạt lịch trình
POST	/api/recurring/run-due	JWT	Header: Bearer Token	Kích hoạt quét và tự động sinh giao dịch trong Transaction
GET	/api/activity	JWT	Query: { page, limit, action }	Truy vấn nhật ký kiểm toán hành vi hệ thống (Audit Logs)
GET	/health	None	None	Healthcheck kiểm tra tình trạng sẵn sàng của AI FastAPI
POST	/classify-expense	None	{ description: String }	Endpoint suy luận phân loại NLP cho một câu văn giao dịch đơn lẻ
POST	/classify-batch	None	{ items: Array[String] }	Endpoint phân loại NLP hàng loạt cho danh sách giao dịch

Phụ lục 4. Cấu hình đầy đủ docker-compose.yml

```

// docker-compose.yml - Multi-container Orchestration
version: '3.8'
services:
  database:
    image: mongo:6.0
    container_name: smart_expense_mongodb
    restart: always
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db

  ai-service:
    build: ./ai-service
    container_name: smart_expense_ai
    restart: always
    ports:
      - "8000:8000"
    environment:
      - PORT=8000

  backend:
    build: ./backend
    container_name: smart_expense_backend
    restart: always
    ports:
      - "5000:5000"
    environment:
      - PORT=5000
      - MONGO_URI=mongodb://database:27017/smart_expense
      - AI_SERVICE_URL=http://ai-service:8000
    depends_on:
      - database

```

- ai-service

frontend:

build: ./frontend

container_name: smart_expense_frontend

restart: always

ports:

- "5173:5173"

depends_on:

- backend

volumes:

mongo_data:

TÀI LIỆU THAM KHẢO

Tiếng Anh

- [1] P. J. Sadalage and M. Fowler, "*NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*". Boston, MA: Addison-Wesley, 2012.
- [2] E. A. Brewer, "Towards Robust Distributed Systems", keynote address, 19th ACM Symposium on Principles of Distributed Computing (PODC), Portland, OR, USA, 2000.
- [3] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm", in Proc. 2014 USENIX Annual Technical Conference (USENIX ATC 14), Philadelphia, PA, USA, 2014, pp. 305-319.
- [4] T. H. Eker, "*Secrets of the Millionaire Mind: Mastering the Inner Game of Wealth*". New York: HarperBusiness, 2005.
- [5] R. Bayer and E. McCreight, "Organization and Maintenance of Large Ordered Indices", Acta Informatica, vol. 1, no. 3, pp. 173-189, 1972.
- [6] MongoDB, Inc., "The ESR (Equality, Sort, Range) Rule", MongoDB Manual. [Online]. Available: <https://www.mongodb.com/docs/manual/tutorial/equality-sort-range-guideline/>
- [7] MongoDB, Inc., "WiredTiger Storage Engine", MongoDB Manual. [Online]. Available: <https://www.mongodb.com/docs/manual/core/wiredtiger/>
- [8] MongoDB, Inc., "\$facet (aggregation stage)", MongoDB Manual. [Online]. Available: <https://www.mongodb.com/docs/manual/reference/operator/aggregation/facet/>
- [9] MongoDB, Inc., "Transactions", MongoDB Manual. [Online]. Available: <https://www.mongodb.com/docs/manual/core/transactions/>
- [10] MongoDB, Inc., "Embedded Data Versus References", MongoDB Manual. [Online]. Available: <https://www.mongodb.com/docs/manual/data-modeling/concepts/embedding-vs-references/>
- [11] MongoDB, Inc., "Shard Keys", MongoDB Manual. [Online]. Available: <https://www.mongodb.com/docs/manual/core/sharding-shard-key/>
- [12] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme", in Proc. 1999 USENIX Annual Technical Conference, Monterey, CA, USA, 1999.

- [13] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)", IETF RFC 7519, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519.html>
- [14] K. Spärck Jones, "A Statistical Interpretation of Term Specificity and Its Application in Retrieval", *Journal of Documentation*, vol. 28, no. 1, pp. 11-21, 1972.
- [15] A. McCallum and K. Nigam, "A Comparison of Event Models for Naive Bayes Text Classification", in *Proc. AAAI-98 Workshop on Learning for Text Categorization*, Madison, WI, USA, 1998, pp. 41-48.
- [16] V. K. Rohatgi and A. K. Md. E. Saleh, "*An Introduction to Probability and Statistics*", 3rd ed. Hoboken, NJ: Wiley, 2015.
- [17] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey", *ACM Computing Surveys*, vol. 41, no. 3, article 15, 2009.
- [18] R. T. Fielding, "*Architectural Styles and the Design of Network-based Software Architectures*", Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000.
- [19] R. Fielding and J. Reschke, Eds., "HTTP Semantics", IETF RFC 9110, Jun. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9110.html>

